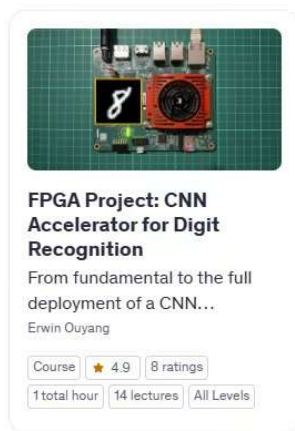


Part 8: Hardware Accelerator for Neural Networks

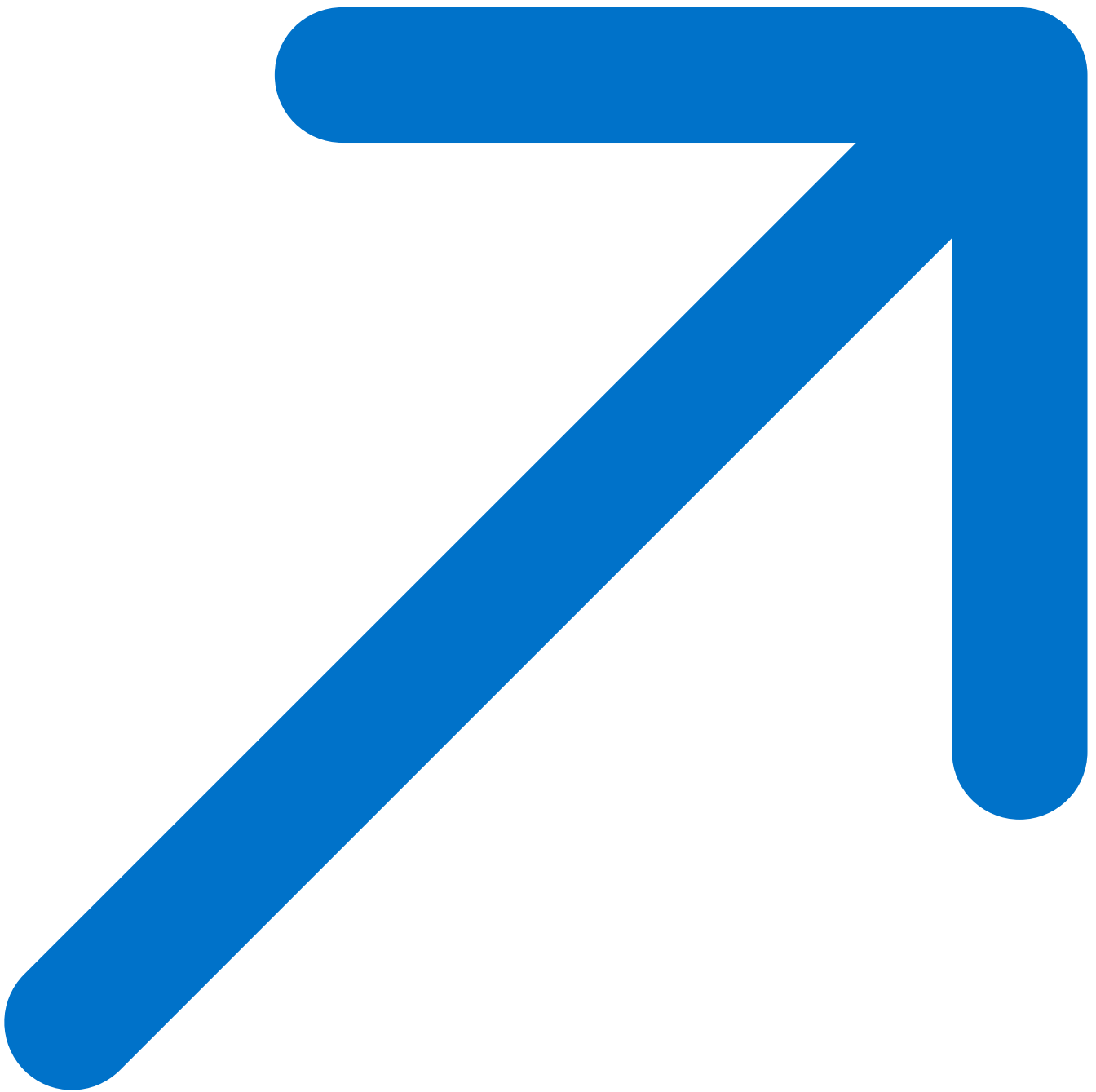
This tutorial contains information on how to build a neural network accelerator. The design process starts with modeling using MATLAB, then building RTL modules, and finally integration with the SoC.

This repository contains all of the code required in order to follow this tutorial.

This **project-based online course** offers **practical insights** into designing **AI accelerators**, specifically a **CNN algorithm** for **handwritten digit classification**. The focus of the course is on the **system design level**, showing how to **integrate a CNN module (written in Verilog RTL) with an application processor running Linux**. The final result of this project is a **web application** for taking a handwritten digit and sending this data to be processed with the **CNN accelerator on the FPGA**. On average, a **speedup factor of 12x** is achieved by using this accelerator compared to the CPU.



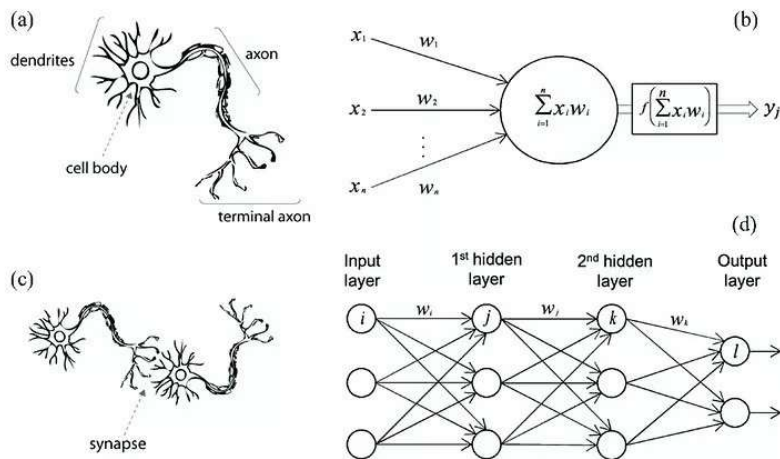
FPGA Project: CNN Accelerator for Digit Recognition: <https://www.udemy.com/course/fpga-project-cnn-accelerator-for-digit-recognition/?referralCode=60E47BBAD02232833118>



In machine learning, a neural network (also called an artificial neural network, abbreviated ANN or NN) is a mathematical model inspired by the structure and function of biological neural networks in human brains.

A NN consists of connected units or nodes called artificial neurons, which loosely model the neurons in a brain. Figure 1(a) shows a neuron in the human brain, and Figure 1(b) shows an artificial neuron. An artificial neuron consists of inputs x , weights w , and an output y .

In the human brain, a neuron can connect to more than one neuron as shown in Figure 1(c). This is the same for the artificial neuron in a NN as shown in Figure 1(d). A NN consists of multiple layers, and each layer consists of multiple neurons.



For every neuron in NN, it does a mathematical computation, expressed as the following equation.

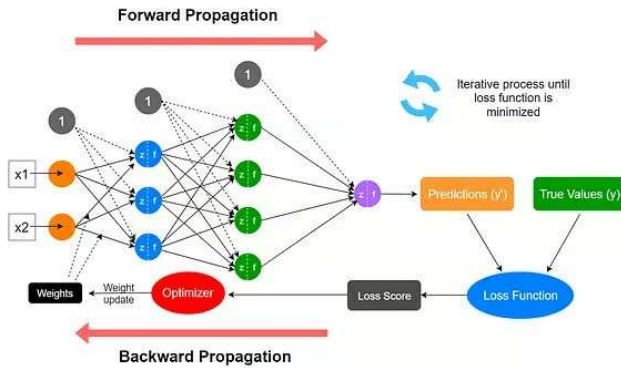
$$y_j = f\left(\sum_{i=1}^n x_i w_i\right)$$

$$y_j = f\left(\sum_{i=1}^n x_i w_i\right)$$

For the whole NN, there are two main steps, which are forward propagation and backward propagation.

- In **forward propagation**, we do the mathematical calculation from the input layer until the output layer to get a prediction. The forward propagation process is also called **inference**.
- In **backward propagation**, we compare the prediction result from the forward propagation process with the true values. Then, we calculate the loss score using a loss function. After that, we use this loss score to update the weight using an optimizer. The back propagation process is also called **training**.

An untrained NN starts with random weights and can't make any predictions. So, the goal of training is to obtain trained weights that can predict the output correctly in the inference process.



In this tutorial, we are going to focus on how to accelerate the forward propagation process on the FPGA as a matrix multiplication process.

Hardware accelerators are purpose-built designs that accompany a processor for accelerating a specific computations. Since processors are designed to handle a wide range of workloads, processor architectures are rarely the most optimal for specific computations.

One example of a hardware accelerator for NN is the Google Tensor Processing Unit (TPU) as shown in Figure 3. TPU is an accelerator application-specific integrated circuit (ASIC) for NN, using Google's own TensorFlow software.

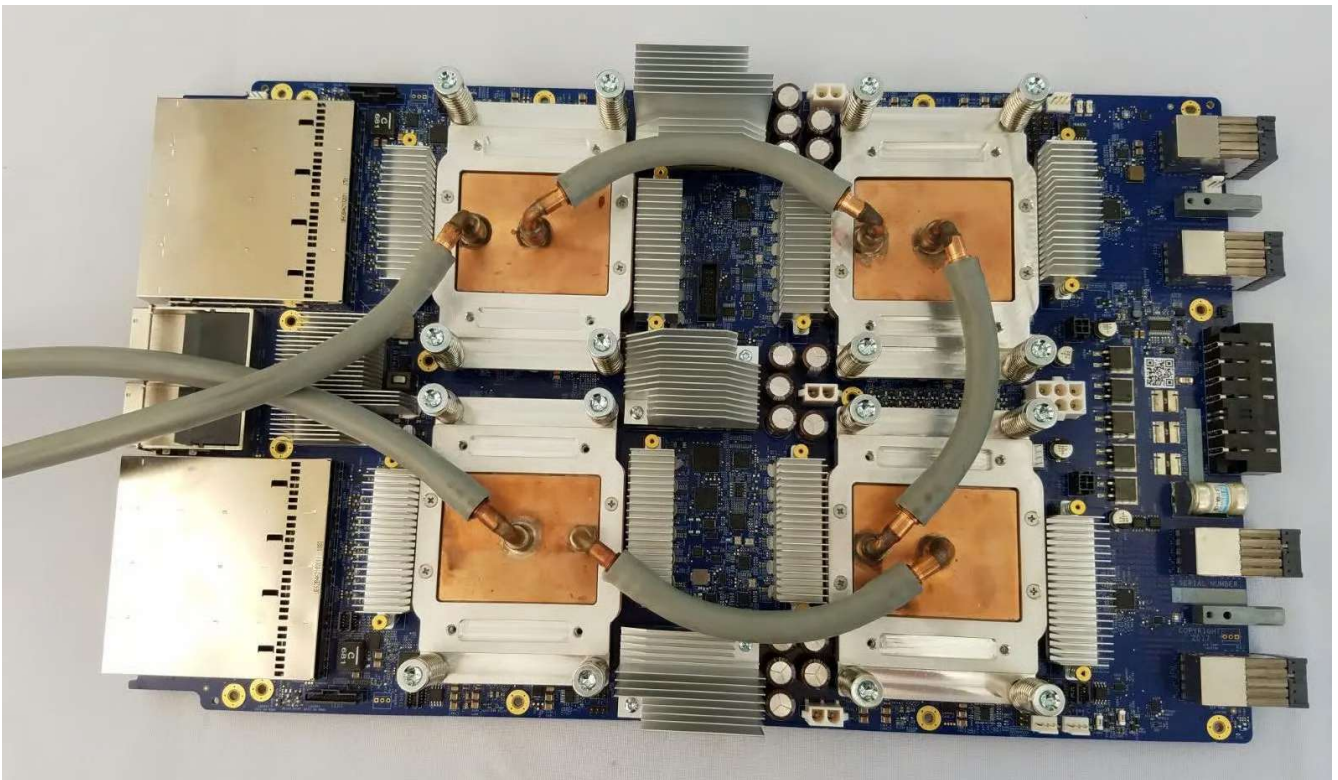
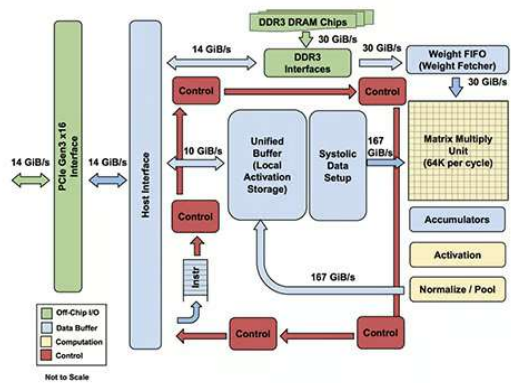


Figure 4 shows the block diagram of the TPU. Its main processing unit is a matrix multiplication unit. It uses a systolic array mechanism that contains 256×256 processing elements (total 65536 ALUs). In this tutorial, we are going to do something similar in concept to this TPU on a smaller scale, with only a 4×4 systolic array.



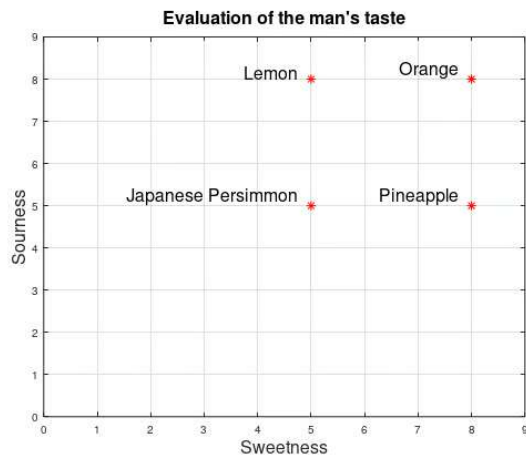
In this tutorial, we are going to use an example of a simple NN model from this [reference](#)



. Let's consider the following example:

- There are four types of fruits: orange, lemon, pineapple, and Japanese persimmon.

- A man eats these four types of fruits, and decides the level of sweetness and sourness of the fruits from the range of 0 to 9.
- After deciding the level of sweetness k_1 and sourness k_2 , then he decides which fruits he likes and which fruits he dislikes.
- So let's consider the fruits he likes as $[t_1, t_2] = [1, 0]$ and the fruits he dislikes as $[t_1, t_2] = [0, 1]$.



Fruits

Sweetness

Sourness

Supervisor

Taste

Orange

8

8

[1,0]

Like

Lemon

8

5

[0,1]

Dislike

Pineapple

5

8

[0,1]

Dislike

Persimmon

5

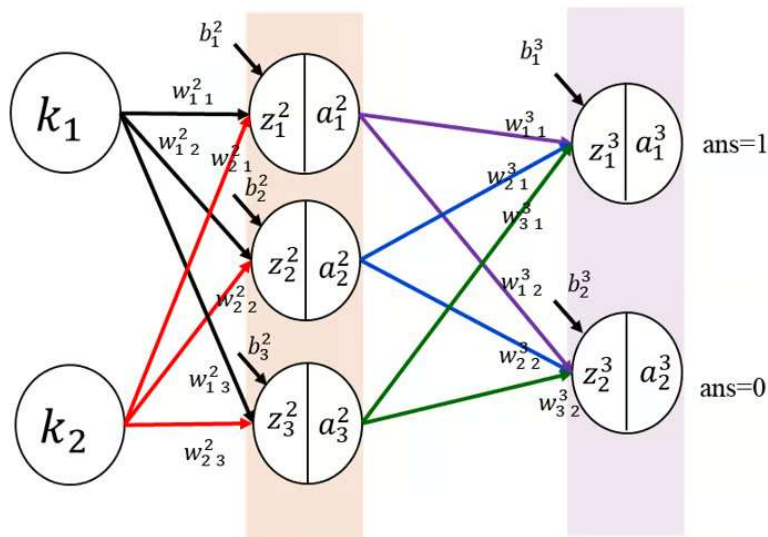
5

[0,1]

Dislike

So, this is a classification problem. The goal is to classify whether the man will like the fruit or not.

In this tutorial, we are only considering the design of **forward propagation in hardware as matrix multiplications**. Figure 5 shows the neural network model with each parameter in its respective layer. It consists of an input layer, a hidden layer, and an output layer.





The definition of each parameter are as describe below:

- k_i is the input signal
- w_{ij}^2 is the weight from input layer to hidden layer
- b_i^2 is the bias for hidden layer
- z_i^2 is the input for hidden layer
- a_i^2 is the output for hidden layer
- w_{ij}^3 is the weight from hidden layer to output layer
- b_i^3 is the bias for output layer
- z_i^3 is the input for output layer
- a_i^3 is the output for output layer

The z_i^2 and z_i^3 is the sum of the bias and product of each weight from input signal and the input signal itself. The calculations are as below:

$$z_i^2 = w_{1i}^2 k_1 + w_{2i}^2 k_2 + b_i^2$$

$$z_i^2 = w_{1i}^2 k_1 + w_{2i}^2 k_2 + b_i^2$$

$$z_i^3 = w_{1i}^3 a_1^2 + w_{2i}^3 a_2^2 + w_{3i}^3 a_3^2 + b_i^3$$

$$z_i^3 = w_{1i}^3 a_1^2 + w_{2i}^3 a_2^2 + w_{3i}^3 a_3^2 + b_i^3$$

The a_i^2 and a_i^3 is the activation function for z_i^2 and z_i^3 , respectively. We can use any functions that can be differentiate and normalize as activation function. For example if we use sigmoid function, the equations for a_i^2 and a_i^3 are as follow:

$$a_i^2 = \sigma(z_i^2) = \frac{1}{1 + e^{-z_i^2}}$$

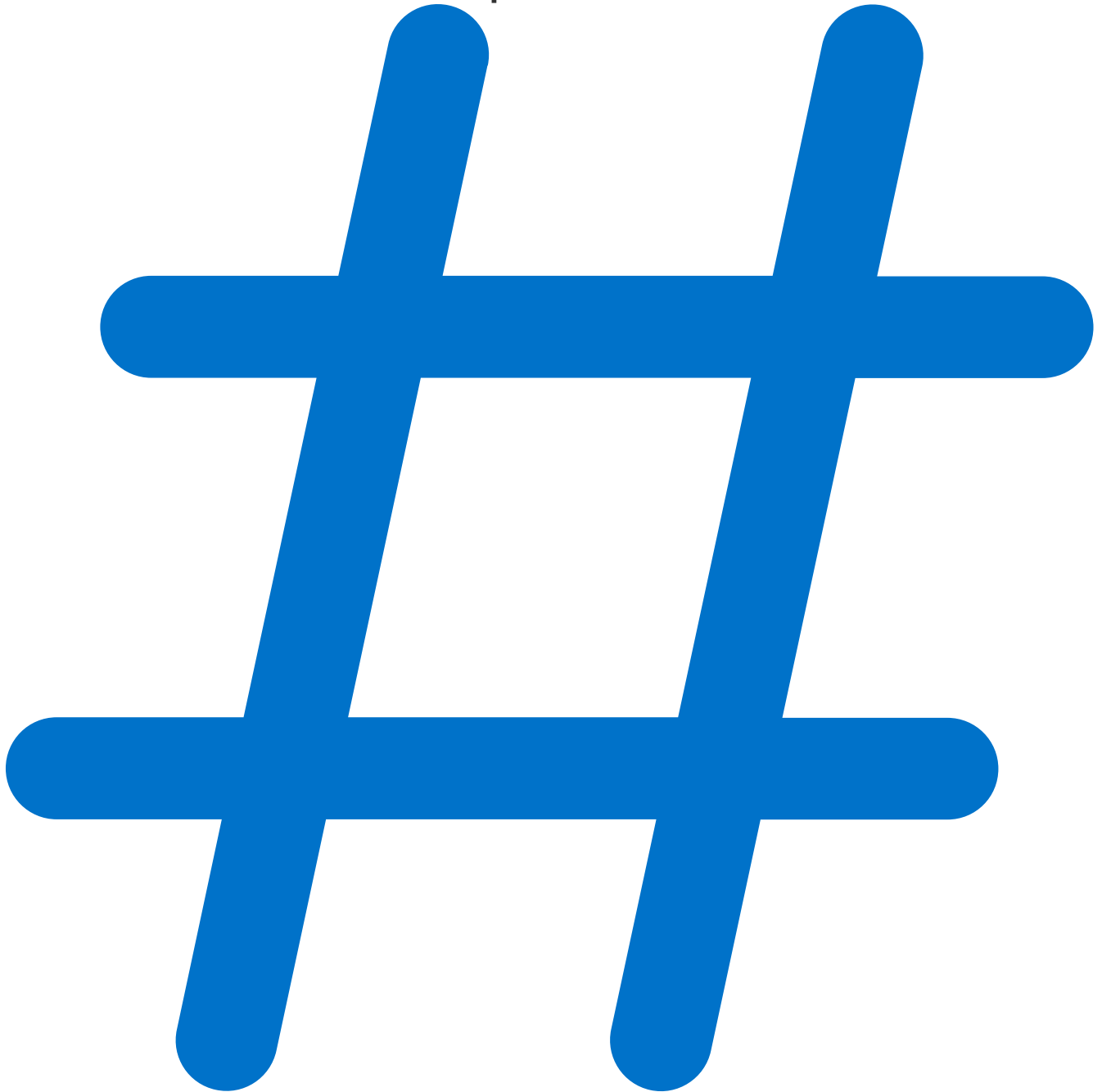
$$a_i^2 = \sigma(z_i^2) = \frac{1}{1 + e^{-z_i^2}}$$

$$a_i^3 = \sigma(z_i^3) = \frac{1}{1 + e^{-z_i^3}}$$

$$a_i^3 = \sigma(z_i^3) = \frac{1}{1 + e^{-z_i^3}}$$

For the backpropagation (training) process, please refer to these file.

2.3. Calculation with Matrix Multiplications



The calculation for every layer can be done at once with matrix operations. The following matrices are constructed from the model in Figure 5. It turns out that the bias values can be included with the weight matrices. But we have to modify the input matrices by adding a new row with a value of all ones.

$$WB_2 = \begin{bmatrix} w_{11}^2 & w_{21}^2 & b_1^2 \\ w_{12}^2 & w_{22}^2 & b_2^2 \\ w_{13}^2 & w_{23}^2 & b_3^2 \end{bmatrix}, WB_3 = \begin{bmatrix} w_{11}^3 & w_{21}^3 & w_{31}^3 & b_1^3 \\ w_{12}^3 & w_{22}^3 & w_{32}^3 & b_2^3 \end{bmatrix}$$

$$WB_2 = \begin{bmatrix} w_{11}^2 & w_{21}^2 & w_{31}^2 & w_{12}^2 & w_{22}^2 & w_{32}^2 & b_{12}^2 & b_{22}^2 & b_{32}^2 \end{bmatrix}, WB_3 = \begin{bmatrix} w_{11}^3 & w_{21}^3 & w_{31}^3 & w_{22}^3 & w_{32}^3 & w_{13}^3 & w_{23}^3 & w_{33}^3 & b_{13}^3 & b_{23}^3 \end{bmatrix}$$

The following are the trained weight values (W_2 and W_3), bias values (B_2 and B_3) obtained from the MATLAB program after training, and input values (K).

$$WB_2 = \begin{bmatrix} 1.37 & 1.37 & -19.88 \\ 0.77 & 0.97 & -0.90 \\ 1.05 & 0.64 & -0.89 \end{bmatrix}, WB_3 = \begin{bmatrix} 7.11 & -1.31 & 0.08 & -2.59 \\ -7.10 & 1.63 & 1.97 & 0.20 \end{bmatrix}$$

$$WB_2 = [1.37 \ 0.77 \ 1.05 \ 1.37 \ 0.97 \ 0.64 \ -19.88 \ -0.90 \ -0.89], WB_3 = [7.11 \ -7.10 \ -1.31 \ 1.63 \ 0.08 \ 1.97 \ -2.59 \ 0.20]$$

$$K = \begin{bmatrix} 8 & 8 & 5 & 5 \\ 8 & 5 & 8 & 5 \end{bmatrix}$$

$$K = [8 \ 8 \ 5 \ 5 \ 8 \ 5 \ 8 \ 5]$$

Padding the input with 1.

$$K_p = \begin{bmatrix} 8 & 8 & 5 & 5 \\ 8 & 5 & 8 & 5 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$K_p = [8 \ 8 \ 1 \ 8 \ 5 \ 1 \ 5 \ 8 \ 1 \ 5 \ 5 \ 1 \ 1 \ 1 \ 1 \ 1]$$

The following are the calculations for the hidden layer.

$$Z_2 = WB_2 * K_p$$

$$Z_2 = WB_2 * K_p$$

$$Z_2 = \begin{bmatrix} 1.37 & 1.37 & -19.88 \\ 0.77 & 0.97 & -0.90 \\ 1.05 & 0.64 & -0.89 \end{bmatrix} * \begin{bmatrix} 8 & 8 & 5 & 5 \\ 8 & 5 & 8 & 5 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$Z_2 = [1.37 \ 0.77 \ 1.05 \ 1.37 \ 0.97 \ 0.64 \ -19.88 \ -0.90 \ -0.89] * [8 \ 8 \ 1 \ 8 \ 5 \ 1 \ 5 \ 8 \ 1 \ 5 \ 5 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$Z_2 = \begin{bmatrix} 2.04 & -2.07 & -2.07 & -6.18 \\ 13.02 & 10.11 & 10.71 & 7.80 \\ 12.63 & 10.71 & 9.48 & 7.56 \end{bmatrix}$$

$$Z_2 = [2.04 \ 13.02 \ 12.63 \ -2.07 \ 10.11 \ 10.71 \ -2.07 \ 10.71 \ 9.48 \ -6.18 \ 7.80 \ 7.56]$$

$$A_2 = \frac{1}{(1 + e^{-Z_2})} = \begin{bmatrix} 0.884 & 0.112 & 0.112 & 0.002 \\ 1.000 & 0.999 & 0.999 & 0.999 \\ 1.000 & 0.999 & 0.999 & 0.999 \end{bmatrix}$$

$$A_2 = (1 + e^{-Z_2})^{-1} = [0.884 \ 1.000 \ 1.000 \ 0.112 \ 0.999 \ 0.999 \ 0.112 \ 0.999 \ 0.999 \ 0.002 \ 0.999 \ 0.999]$$

Padding the A_2 with 1.

$$A_{2p} = \begin{bmatrix} 0.884 & 0.112 & 0.112 & 0.002 \\ 1.000 & 0.999 & 0.999 & 0.999 \\ 1.000 & 0.999 & 0.999 & 0.999 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$A_{2p} = \begin{bmatrix} 0.884 & 1.000 & 1.000 & 10.112 & 0.999 & 0.999 & 10.112 & 0.999 & 0.999 & 10.002 & 0.999 & 0.999 & 1 \end{bmatrix}$$

The following are the calculations for the output layer.

$$Z_3 = WB_3 * A_{2p}$$

$$Z_3 = WB_3 * A_{2p}$$

$$Z_3 = \begin{bmatrix} 7.11 & -1.31 & 0.08 & -2.59 \\ -7.10 & 1.63 & 1.97 & 0.20 \end{bmatrix} * \begin{bmatrix} 0.884 & 0.112 & 0.112 & 0.002 \\ 1.000 & 0.999 & 0.999 & 0.999 \\ 1.000 & 0.999 & 0.999 & 0.999 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$Z_3 = [7.11 \ -7.10 \ -1.31 \ 1.63 \ 0.08 \ 1.97 \ -2.59 \ 0.20] * \begin{bmatrix} 0.884 & 1.000 & 1.000 & 10.112 & 0.999 & 0.999 & 10.112 & 0.999 & 0.999 & 10.002 & 0.999 & 0.999 & 1 \end{bmatrix}$$

$$Z_3 = \begin{bmatrix} 2.47 & -3.02 & -3.02 & -3.80 \\ -2.48 & 3.00 & 3.00 & 3.78 \end{bmatrix}$$

$$Z_3 = [2.47 \ -2.48 \ -3.02 \ 3.00 \ -3.02 \ 3.00 \ -3.80 \ 3.78]$$

$$A_3 = \frac{1}{(1 + e^{-Z_2})} = \begin{bmatrix} 0.922 & 0.046 & 0.046 & 0.021 \\ 0.077 & 0.952 & 0.952 & 0.977 \end{bmatrix}$$

$$A_3 = (1 + e^{-Z_2})^{-1} = [0.922 \ 0.077 \ 0.046 \ 0.952 \ 0.046 \ 0.952 \ 0.021 \ 0.977]$$

The following is the MATLAB code for the calculations. You can run this file on the online MATLAB or Octave compiler.

```
k = [8, 8, 5, 5;
      8, 5, 8, 5]
wb2 = [1.37, 1.37, -19.88;
        0.77, 0.97, -0.90;
        1.05, 0.64, -0.89]
wb3 = [ 7.11, -1.31, 0.08, -2.59;
        -7.10, 1.63, 1.97, 0.20]

k_padded = cat(1, k, ones(1, 4))

z2 = wb2 * k_padded
a2 = 1./(1+exp(-z2))

a2_padded = cat(1, a2, ones(1, 4))

z3 = wb3 * a2_padded
```

```
a3 = 1./(1+exp(-z3))
```

```
a3_rounded = round(a3)
```

```
k =
```

8	8	5	5
8	5	8	5

```
wb2 =
```

1.3700	1.3700	-19.8800
0.7700	0.9700	-0.9000
1.0500	0.6400	-0.8900

```
wb3 =
```

7.110000	-1.310000	0.080000	-2.590000
-7.100000	1.630000	1.970000	0.200000

```
k_padded =
```

8	8	5	5
8	5	8	5
1	1	1	1

```
z2 =
```

2.0400	-2.0700	-2.0700	-6.1800
13.0200	10.1100	10.7100	7.8000
12.6300	10.7100	9.4800	7.5600

```
a2 =
```

8.8493e-01	1.1205e-01	1.1205e-01	2.0662e-03
1.0000e+00	9.9996e-01	9.9998e-01	9.9959e-01
1.0000e+00	9.9998e-01	9.9992e-01	9.9948e-01

```
a2_padded =
```

8.8493e-01	1.1205e-01	1.1205e-01	2.0662e-03
1.0000e+00	9.9996e-01	9.9998e-01	9.9959e-01
1.0000e+00	9.9998e-01	9.9992e-01	9.9948e-01
1.0000e+00	1.0000e+00	1.0000e+00	1.0000e+00

```
z3 =
```

2.4719	-3.0233	-3.0233	-3.8048
-2.4830	3.0044	3.0043	3.7836

a3 =

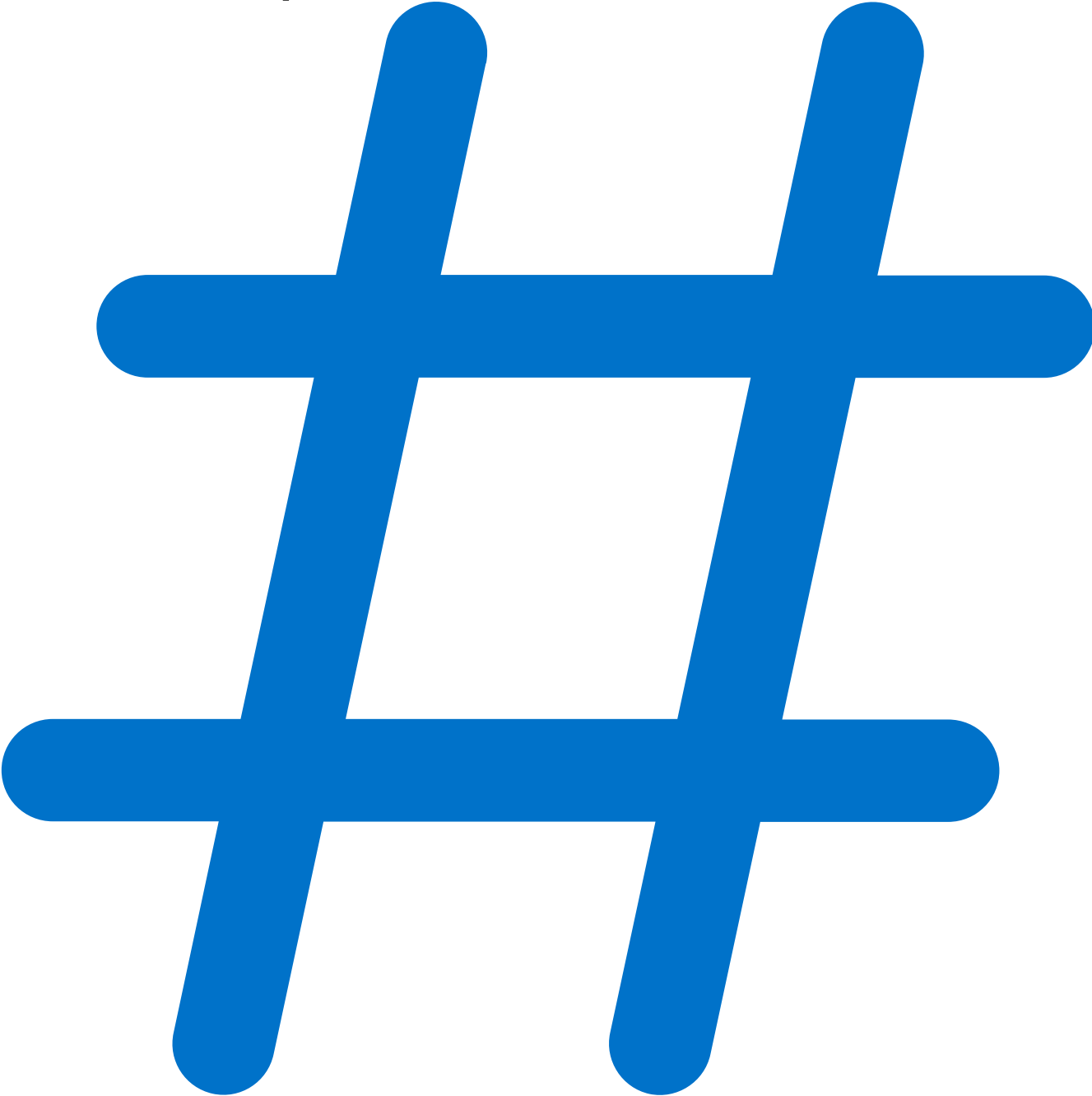
0.922147	0.046385	0.046383	0.021778
0.077056	0.952771	0.952767	0.977766

a3_rounded =

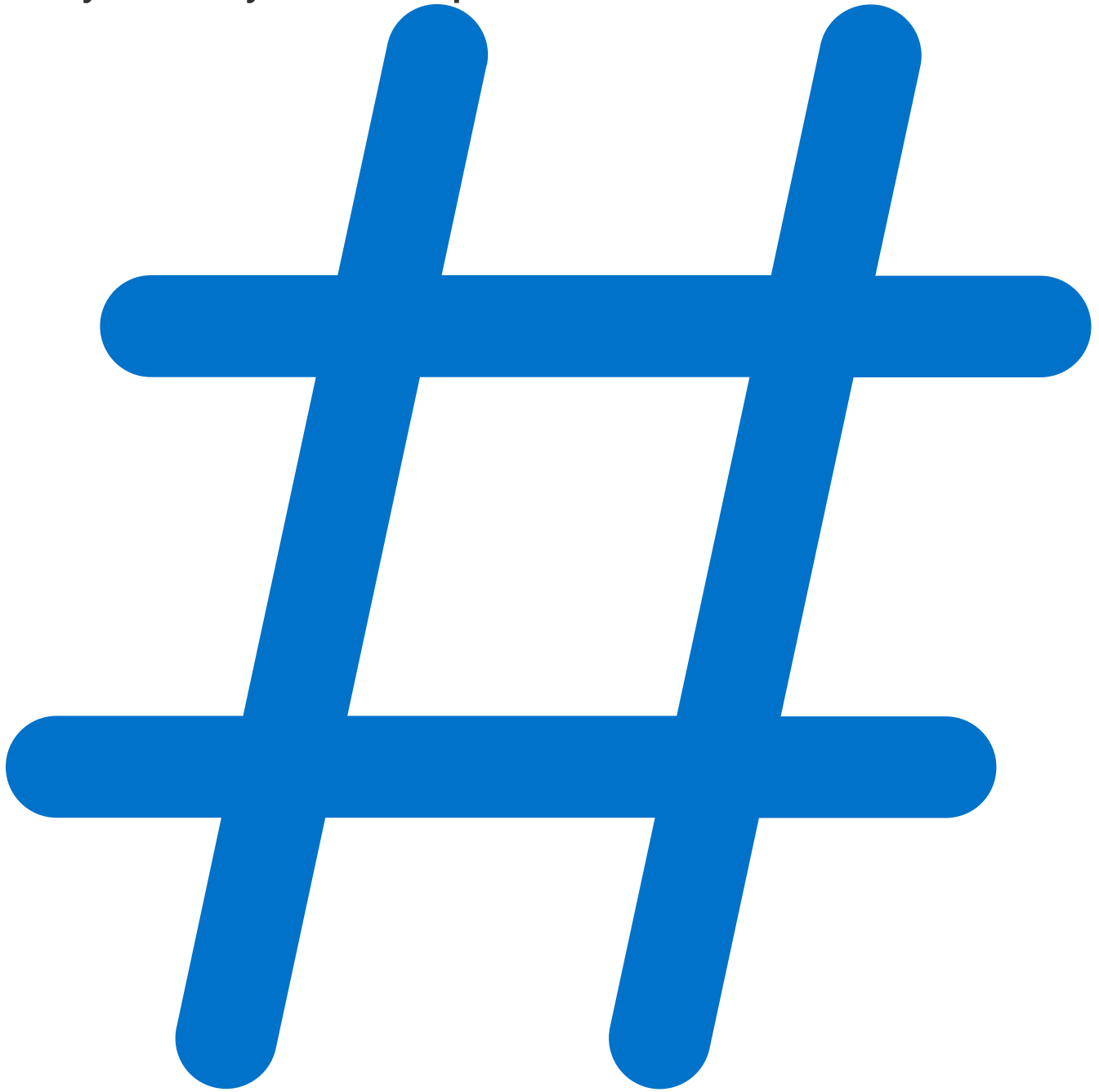
1	0	0	0
0	1	1	1

The result of a3_rounded is the same as the result in the previous table.

3. Matrix Multiplication Module



3.1. Systolic Array Matrix Multiplication



The multiplication of matrices is a very common operation in engineering and scientific problems. The sequential implementation of this operation is very time consuming for large matrices. In hardware design, there is an algorithm for processing matrix multiplication using a systolic array. The 2D systolic array forms the heart of the Matrix Multiplier Unit (MXU) on the Google TPU and the new deep learning FPGAs from Xilinx.

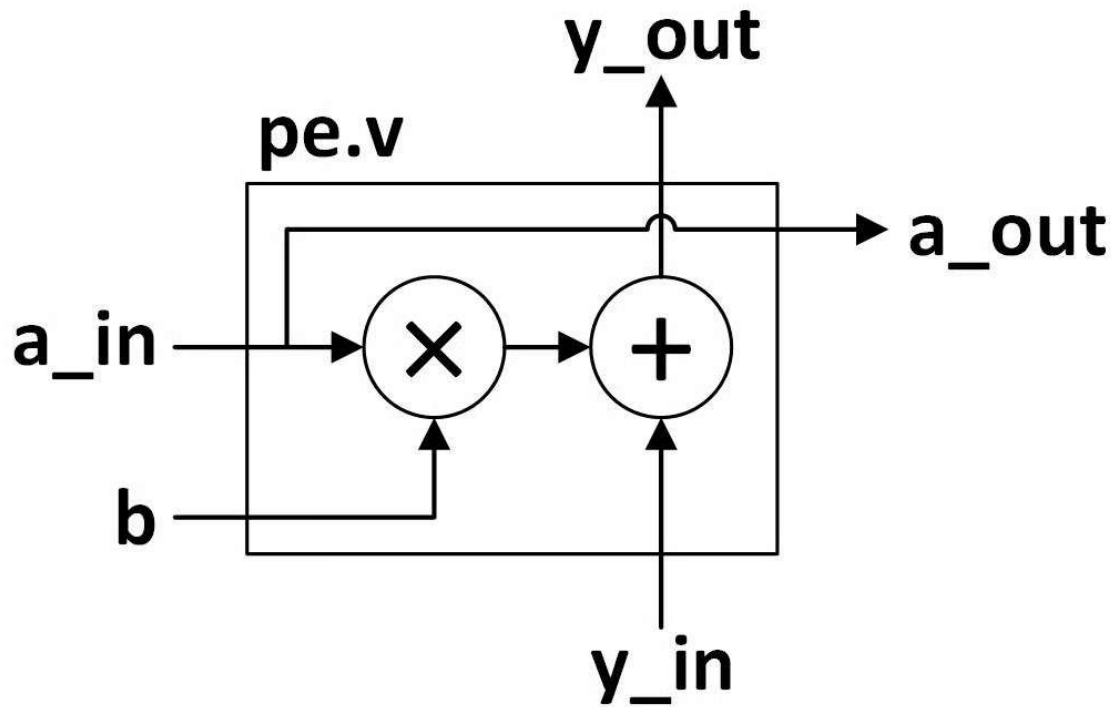


Figure 6. A processing element (PE)

A systolic array consists of multiple processing elements (PE) and registers. A PE consists of a multiplier and an adder as show in Figure 6. An example of 4x4 systolic array is shown in Figure 7. This illustration shows how to multiply two 4x4 matrices, AA and BB .

$$Y = A * B = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} = \begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix}$$

$$Y=A*B= \begin{bmatrix} 159 & 132 & 610 & 143 & 711 & 115 & 481 & 216 \end{bmatrix} * \begin{bmatrix} 159 & 132 & 610 & 143 & 711 & 115 & 481 & 216 \end{bmatrix} = \begin{bmatrix} 90 & 202 & 314 & 426 \end{bmatrix}$$

$$\begin{bmatrix} 100 & 228 & 356 & 484 & 110 & 254 & 398 & 542 & 120 & 280 & 440 & 600 \end{bmatrix}$$

The input AA is called **moving input**, and the input BB is called **stationary input**. Every clock cycle, the input AA enter the systolic array diagonally. Then, the output YY come out of the systolic array diagonally for every clock cycle.

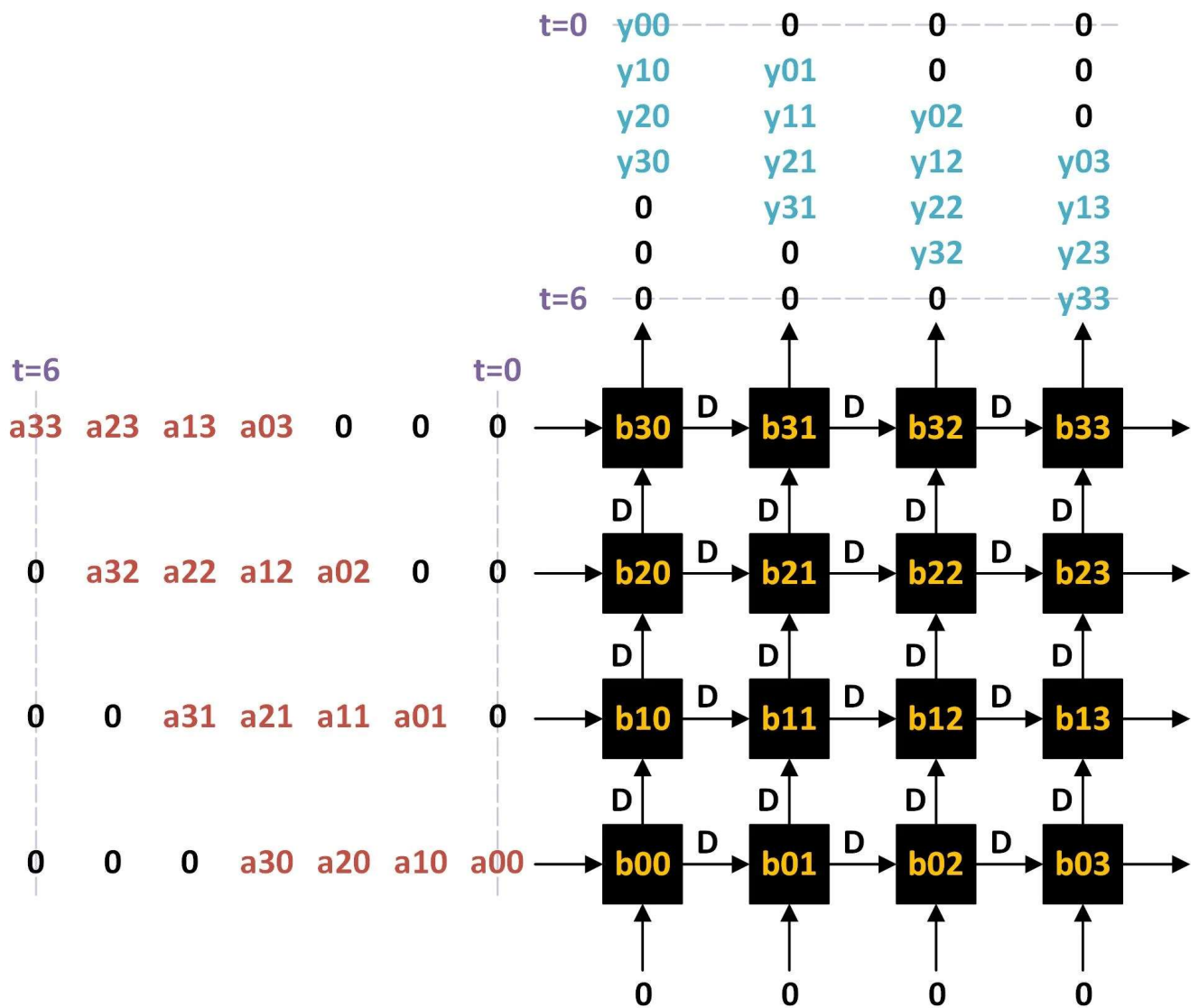


Figure 7. An example of 4x4 systolic array

This animation shows how the systolic array multiplies these 4x4 matrices step-by-step.

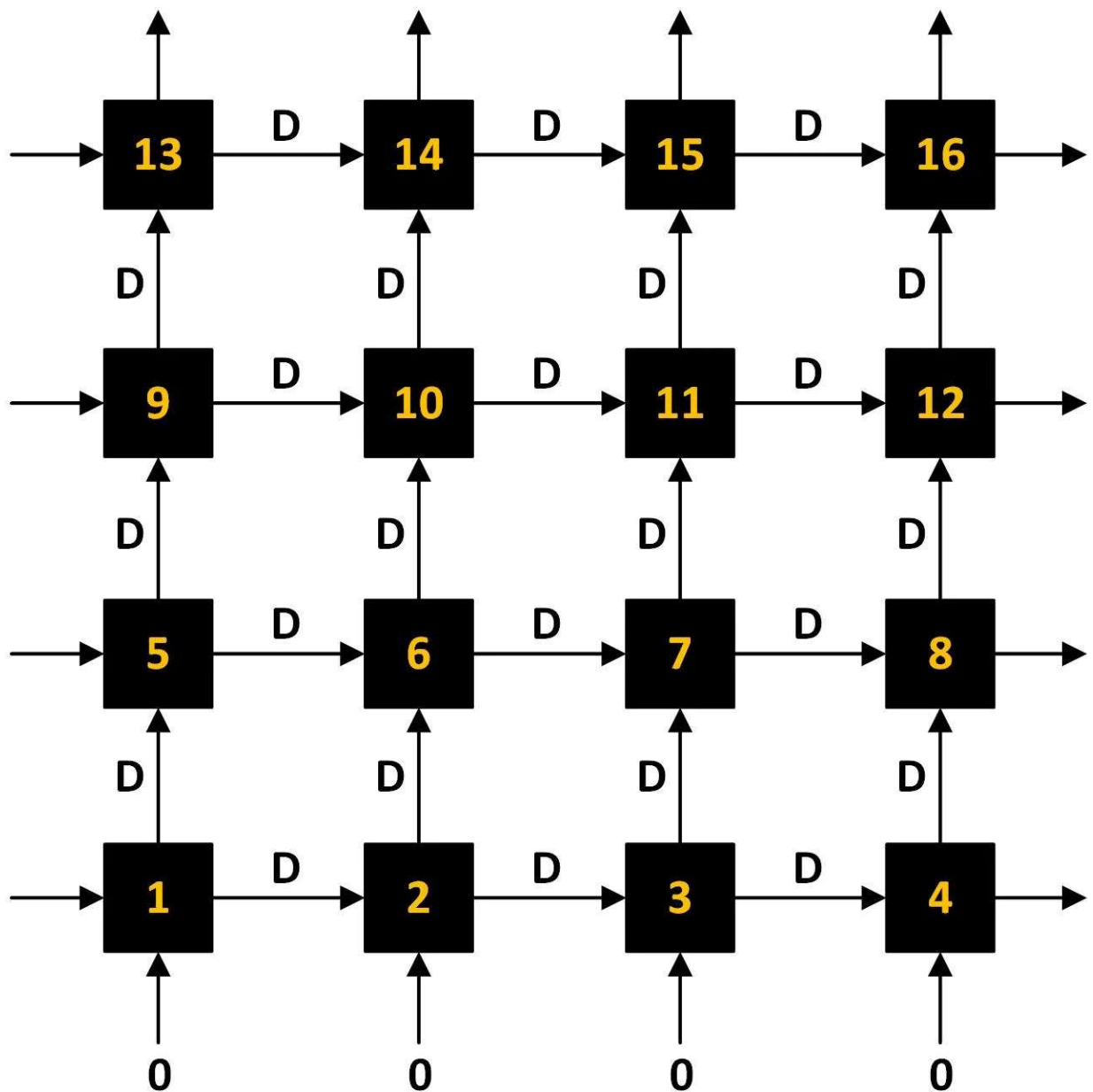


Figure 8. The step-by-step processing of 4x4 matrices in the systolic array

This is the Verilog implementation of the 4x4 systolic array. In this implementation, I also add registers arranged before the input in such a way that the input pattern is no longer diagonal. This is done in order to simplify the control process.

systolic.v

```
`timescale 1ns / 1ps
```

```
module systolic
```

```
  #(
```

```
    parameter WIDTH = 16,
```

```
    parameter FRAC_BIT = 10
```

```
  )
```

```
  (
```

```

    input wire          clk,
    input wire          rst_n,
    input wire          en,
    input wire          clr,
    input wire signed [WIDTH-1:0] a0, a1, a2, a3,
    input wire          in_valid,
    input wire signed [WIDTH-1:0] b00, b01, b02, b03,
    input wire signed [WIDTH-1:0] b10, b11, b12, b13,
    input wire signed [WIDTH-1:0] b20, b21, b22, b23,
    input wire signed [WIDTH-1:0] b30, b31, b32, b33,
    output wire signed [WIDTH-1:0] y0, y1, y2, y3,
    output wire          out_valid
);

// *** Input registers ***
wire signed [WIDTH-1:0] a0_reg0;
wire signed [WIDTH-1:0] a1_reg0, a1_reg1;
wire signed [WIDTH-1:0] a2_reg0, a2_reg1, a2_reg2;
wire signed [WIDTH-1:0] a3_reg0, a3_reg1, a3_reg2, a3_reg3;

// *** a in ***
wire signed [WIDTH-1:0] a00_in, a01_in, a02_in, a03_in,
                        a10_in, a11_in, a12_in, a13_in,
                        a20_in, a21_in, a22_in, a23_in,
                        a30_in, a31_in, a32_in, a33_in;

// *** y in ***
wire signed [WIDTH-1:0] y00_in, y01_in, y02_in, y03_in,
                        y10_in, y11_in, y12_in, y13_in,
                        y20_in, y21_in, y22_in, y23_in,
                        y30_in, y31_in, y32_in, y33_in;

// *** a out ***
wire signed [WIDTH-1:0] a00_out, a01_out, a02_out, a03_out,
                        a10_out, a11_out, a12_out, a13_out,
                        a20_out, a21_out, a22_out, a23_out,
                        a30_out, a31_out, a32_out, a33_out;

// *** y out ***
wire signed [WIDTH-1:0] y00_out, y01_out, y02_out, y03_out,
                        y10_out, y11_out, y12_out, y13_out,
                        y20_out, y21_out, y22_out, y23_out,
                        y30_out, y31_out, y32_out, y33_out;

// *** Output registers ***
wire signed [WIDTH-1:0] y0_tmp, y1_tmp, y2_tmp, y3_tmp;
wire signed [WIDTH-1:0] y0_reg0, y0_reg1, y0_reg2, y0_reg3;
wire signed [WIDTH-1:0] y1_reg0, y1_reg1, y1_reg2;
wire signed [WIDTH-1:0] y2_reg0, y2_reg1;

```

```

wire signed [WIDTH-1:0] y3_reg0;

// *** Valid registers ***
wire in_valid_reg0, in_valid_reg1, in_valid_reg2, in_valid_reg3, in_valid_reg4,
in_valid_reg5, in_valid_reg6, in_valid_reg7, in_valid_reg8;

// *** Input registers for systolic data setup ***
register #(WIDTH) reg_a0_0(clk, rst_n, en, clr, a0,      a0_reg0);
register #(WIDTH) reg_a1_0(clk, rst_n, en, clr, a1,      a1_reg0);
register #(WIDTH) reg_a1_1(clk, rst_n, en, clr, a1_reg0, a1_reg1);
register #(WIDTH) reg_a2_0(clk, rst_n, en, clr, a2,      a2_reg0);
register #(WIDTH) reg_a2_1(clk, rst_n, en, clr, a2_reg0, a2_reg1);
register #(WIDTH) reg_a2_2(clk, rst_n, en, clr, a2_reg1, a2_reg2);
register #(WIDTH) reg_a3_0(clk, rst_n, en, clr, a3,      a3_reg0);
register #(WIDTH) reg_a3_1(clk, rst_n, en, clr, a3_reg0, a3_reg1);
register #(WIDTH) reg_a3_2(clk, rst_n, en, clr, a3_reg1, a3_reg2);
register #(WIDTH) reg_a3_3(clk, rst_n, en, clr, a3_reg2, a3_reg3);

// *** First x inputs ***
assign a00_in = a0_reg0;
assign a10_in = a1_reg1;
assign a20_in = a2_reg2;
assign a30_in = a3_reg3;

// *** First z inputs ***
assign y00_in = 0;
assign y01_in = 0;
assign y02_in = 0;
assign y03_in = 0;

// *** 4x4 systolic array ***
// *** Row 0 from bottom ***
pe #(WIDTH, FRAC_BIT) pe00(a00_in, y00_in, b00, a00_out, y00_out);
pe #(WIDTH, FRAC_BIT) pe01(a01_in, y01_in, b01, a01_out, y01_out);
pe #(WIDTH, FRAC_BIT) pe02(a02_in, y02_in, b02, a02_out, y02_out);
pe #(WIDTH, FRAC_BIT) pe03(a03_in, y03_in, b03, a03_out, y03_out);
// *** Row 1 from bottom ***
pe #(WIDTH, FRAC_BIT) pe10(a10_in, y10_in, b10, a10_out, y10_out);
pe #(WIDTH, FRAC_BIT) pe11(a11_in, y11_in, b11, a11_out, y11_out);
pe #(WIDTH, FRAC_BIT) pe12(a12_in, y12_in, b12, a12_out, y12_out);
pe #(WIDTH, FRAC_BIT) pe13(a13_in, y13_in, b13, a13_out, y13_out);
// *** Row 2 from bottom ***
pe #(WIDTH, FRAC_BIT) pe20(a20_in, y20_in, b20, a20_out, y20_out);
pe #(WIDTH, FRAC_BIT) pe21(a21_in, y21_in, b21, a21_out, y21_out);
pe #(WIDTH, FRAC_BIT) pe22(a22_in, y22_in, b22, a22_out, y22_out);
pe #(WIDTH, FRAC_BIT) pe23(a23_in, y23_in, b23, a23_out, y23_out);

```

```

// *** Row 3 from bottom ***
pe #(WIDTH, FRAC_BIT) pe30(a30_in, y30_in, b30, a30_out, y30_out);
pe #(WIDTH, FRAC_BIT) pe31(a31_in, y31_in, b31, a31_out, y31_out);
pe #(WIDTH, FRAC_BIT) pe32(a32_in, y32_in, b32, a32_out, y32_out);
pe #(WIDTH, FRAC_BIT) pe33(a33_in, y33_in, b33, a33_out, y33_out);

// *** Internal registers ***
// *** Row 0 from bottom ***
register #(WIDTH) reg_a00(clk, rst_n, en, clr, a00_out, a01_in);
register #(WIDTH) reg_a01(clk, rst_n, en, clr, a01_out, a02_in);
register #(WIDTH) reg_a02(clk, rst_n, en, clr, a02_out, a03_in);
// *** Row 1 from bottom ***
register #(WIDTH) reg_a10(clk, rst_n, en, clr, a10_out, a11_in);
register #(WIDTH) reg_a11(clk, rst_n, en, clr, a11_out, a12_in);
register #(WIDTH) reg_a12(clk, rst_n, en, clr, a12_out, a13_in);
// *** Row 2 from bottom ***
register #(WIDTH) reg_a20(clk, rst_n, en, clr, a20_out, a21_in);
register #(WIDTH) reg_a21(clk, rst_n, en, clr, a21_out, a22_in);
register #(WIDTH) reg_a22(clk, rst_n, en, clr, a22_out, a23_in);
// *** Row 3 from bottom ***
register #(WIDTH) reg_a30(clk, rst_n, en, clr, a30_out, a31_in);
register #(WIDTH) reg_a31(clk, rst_n, en, clr, a31_out, a32_in);
register #(WIDTH) reg_a32(clk, rst_n, en, clr, a32_out, a33_in);
// *** Column 0 from left ***
register #(WIDTH) reg_y00(clk, rst_n, en, clr, y00_out, y10_in);
register #(WIDTH) reg_y10(clk, rst_n, en, clr, y10_out, y20_in);
register #(WIDTH) reg_y20(clk, rst_n, en, clr, y20_out, y30_in);
register #(WIDTH) reg_y30(clk, rst_n, en, clr, y30_out, y0_tmp);
// *** Column 1 from left ***
register #(WIDTH) reg_y01(clk, rst_n, en, clr, y01_out, y11_in);
register #(WIDTH) reg_y11(clk, rst_n, en, clr, y11_out, y21_in);
register #(WIDTH) reg_y21(clk, rst_n, en, clr, y21_out, y31_in);
register #(WIDTH) reg_y31(clk, rst_n, en, clr, y31_out, y1_tmp);
// *** Column 2 from left ***
register #(WIDTH) reg_y02(clk, rst_n, en, clr, y02_out, y12_in);
register #(WIDTH) reg_y12(clk, rst_n, en, clr, y12_out, y22_in);
register #(WIDTH) reg_y22(clk, rst_n, en, clr, y22_out, y32_in);
register #(WIDTH) reg_y32(clk, rst_n, en, clr, y32_out, y2_tmp);
// *** Column 3 from left ***
register #(WIDTH) reg_y03(clk, rst_n, en, clr, y03_out, y13_in);
register #(WIDTH) reg_y13(clk, rst_n, en, clr, y13_out, y23_in);
register #(WIDTH) reg_y23(clk, rst_n, en, clr, y23_out, y33_in);
register #(WIDTH) reg_y33(clk, rst_n, en, clr, y33_out, y3_tmp);

// *** Output registers ***
register #(WIDTH) reg_y0_0(clk, rst_n, en, clr, y0_tmp, y0_reg0);

```

```

register #(WIDTH) reg_y0_1(clk, rst_n, en, clr, y0_reg0, y0_reg1);
register #(WIDTH) reg_y0_2(clk, rst_n, en, clr, y0_reg1, y0_reg2);
register #(WIDTH) reg_y0_3(clk, rst_n, en, clr, y0_reg2, y0_reg3);
register #(WIDTH) reg_y1_0(clk, rst_n, en, clr, y1_tmp, y1_reg0);
register #(WIDTH) reg_y1_1(clk, rst_n, en, clr, y1_reg0, y1_reg1);
register #(WIDTH) reg_y1_2(clk, rst_n, en, clr, y1_reg1, y1_reg2);
register #(WIDTH) reg_y2_0(clk, rst_n, en, clr, y2_tmp, y2_reg0);
register #(WIDTH) reg_y2_1(clk, rst_n, en, clr, y2_reg0, y2_reg1);
register #(WIDTH) reg_y3_0(clk, rst_n, en, clr, y3_tmp, y3_reg0);

// *** Valid registers ***
register #(1) reg_valid_0(clk, rst_n, en, clr, in_valid, in_valid_reg0);
register #(1) reg_valid_1(clk, rst_n, en, clr, in_valid_reg0, in_valid_reg1);
register #(1) reg_valid_2(clk, rst_n, en, clr, in_valid_reg1, in_valid_reg2);
register #(1) reg_valid_3(clk, rst_n, en, clr, in_valid_reg2, in_valid_reg3);
register #(1) reg_valid_4(clk, rst_n, en, clr, in_valid_reg3, in_valid_reg4);
register #(1) reg_valid_5(clk, rst_n, en, clr, in_valid_reg4, in_valid_reg5);
register #(1) reg_valid_6(clk, rst_n, en, clr, in_valid_reg5, in_valid_reg6);
register #(1) reg_valid_7(clk, rst_n, en, clr, in_valid_reg6, in_valid_reg7);
register #(1) reg_valid_8(clk, rst_n, en, clr, in_valid_reg7, in_valid_reg8);

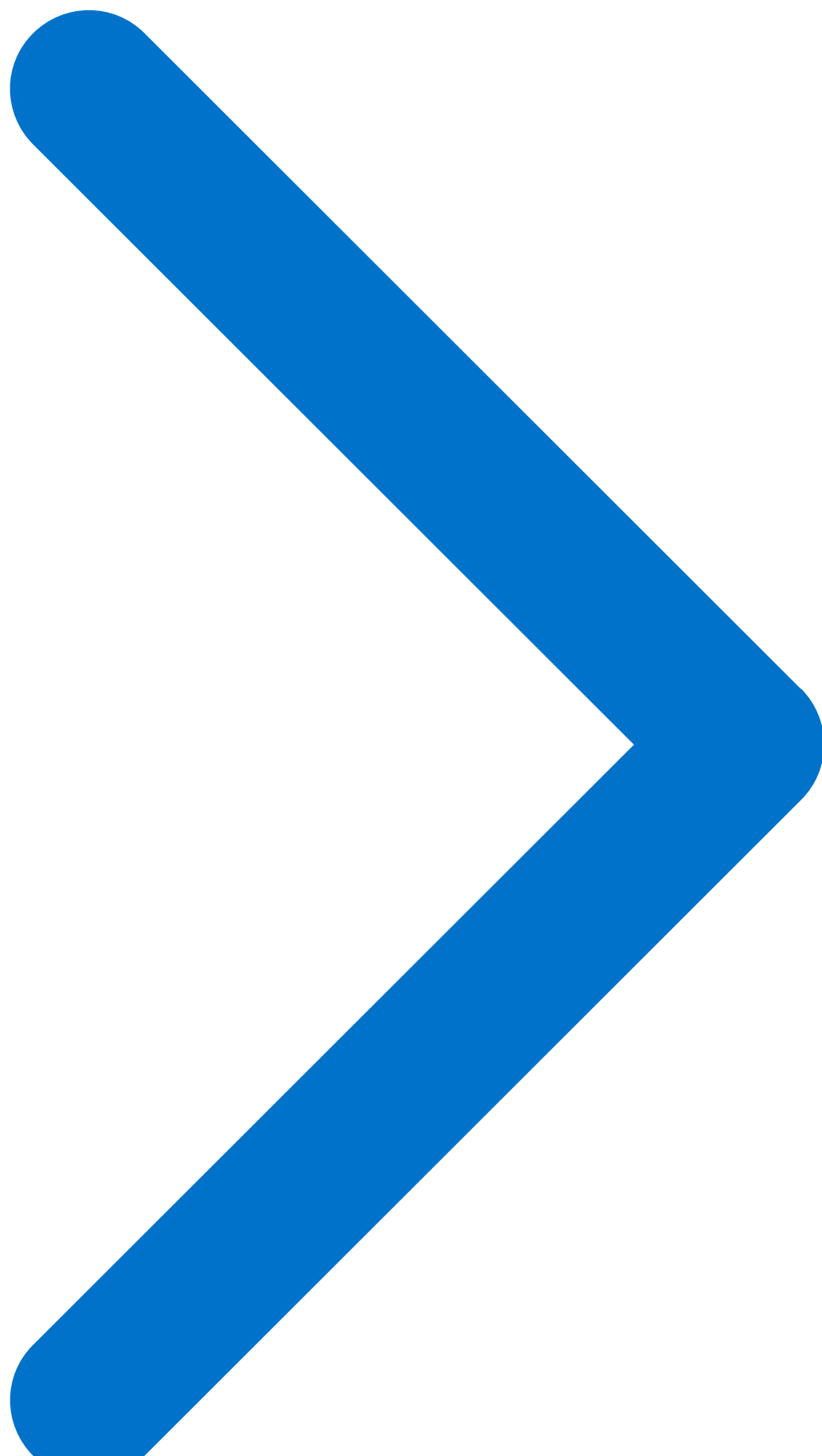
// *** Outputs ***
assign y0 = y0_reg3;
assign y1 = y1_reg2;
assign y2 = y2_reg1;
assign y3 = y3_reg0;
assign out_valid = in_valid_reg8;

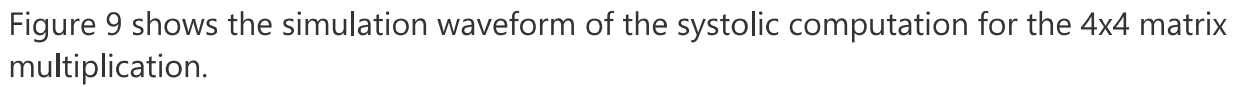
```

endmodule

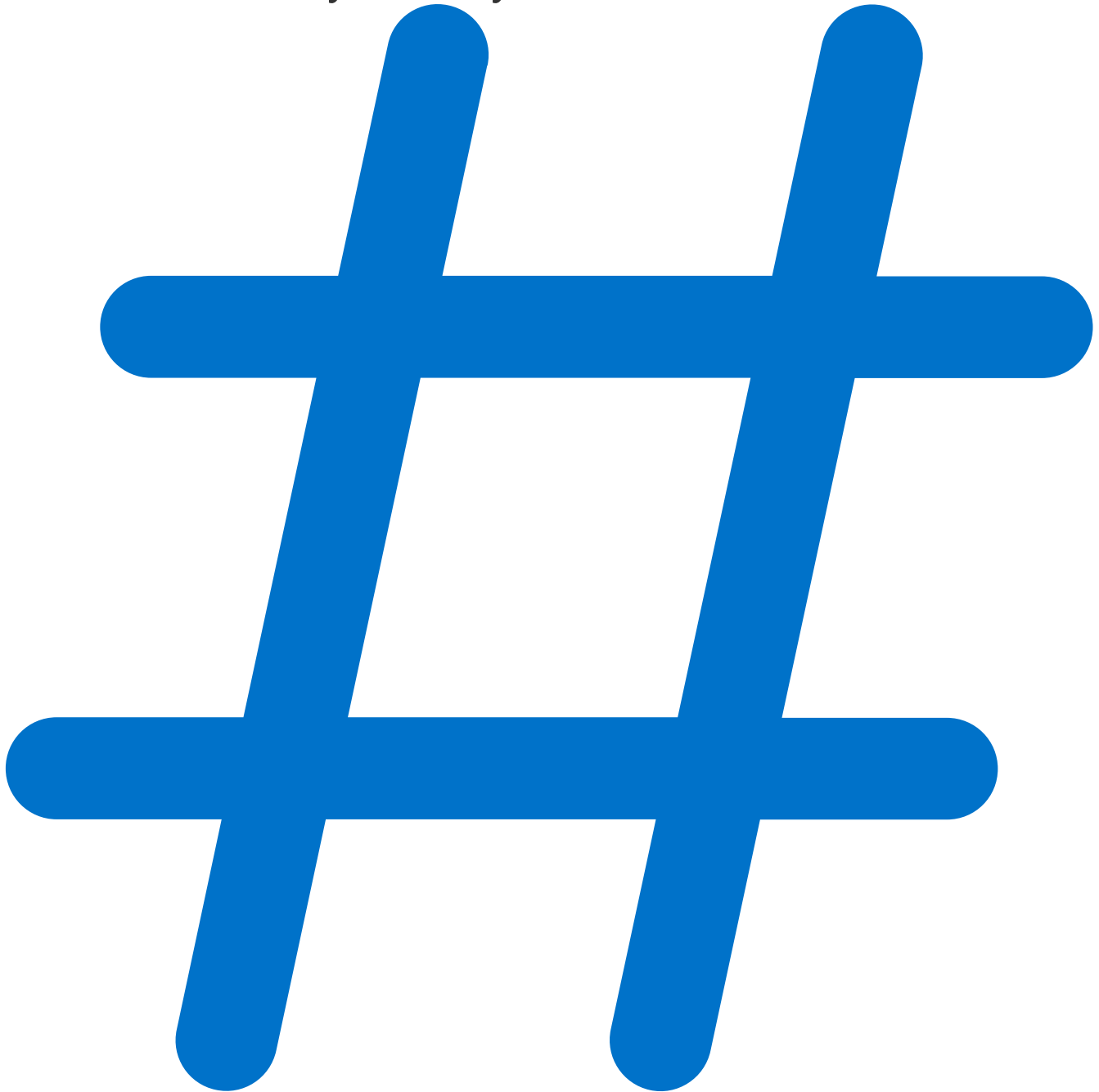
To test the systolic module, we can use this testbench.

https://github.com/weenslab/fpga101/blob/main/part_8/verilog/testbench/systolic_tb.vgithub.com





3.2. Process NN with Systolic Array



Our NN works with decimal numbers. In the hardware, we are going to use fixed-point representation to represent the decimal number. The Q notation is used to specify the parameters of a binary fixed-point number format.

In this design, we use **Q5.10**. It means that the fixed-point numbers have:

- **5 bits** for the integer part,
- **10 bits** for the fraction part, and
- **1 bit** for the sign.

So, the total number of bits is 16-bit.

In the Verilog module for the systolic, we can change how many fraction bits the module works with by changing the `FRAC_BIT` parameter.

We can test the systolic module with the real matrix value from the NN model with this testbench.

Here is the result of matrix multiplication for the hidden layer $Z_2 = WB_2 * K_p$.

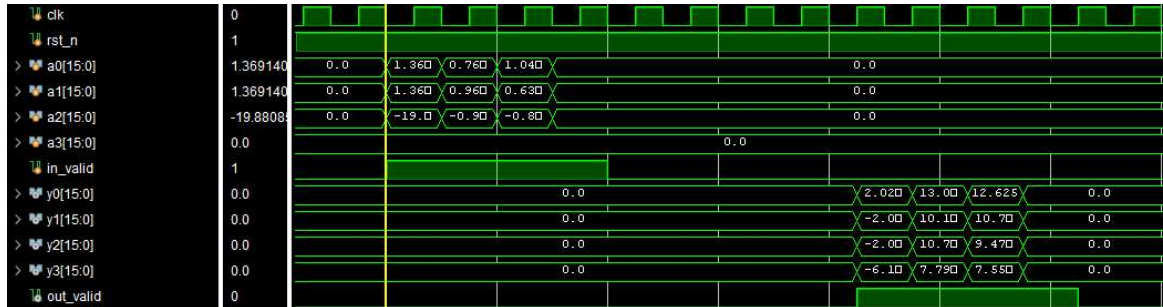


Figure 10. Simulation waveform for the hidden layer

Then, this is the result of matrix multiplication for the output layer $Z_3 = WB_3 * A_{2p}$.

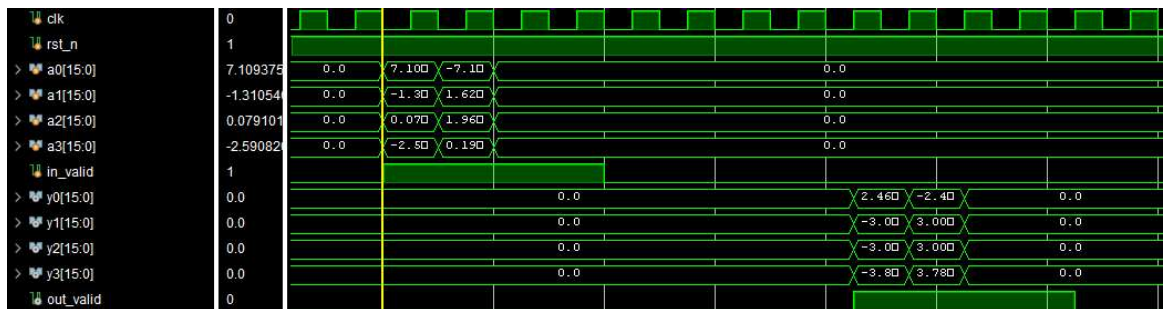


Figure 11. Simulation waveform for the output layer

To calculate the sigmoid function, we use approximation with the look-up table (LUT) module. The module can calculate sigmoid with inputs ranging from -8.00000 to 7.93750. Any input outside this range will be saturated to the maximum input within this range. This is the sigmoid LUT module in Verilog.

Now, we already have systolic and sigmoid blocks. The next step is to connect these blocks to the memory input and output. We also need a control module to control the flow of data in this system. This is how the data flows:

1. Input data are read from the **BRAM input**, then sent to the **stationary input** of the systolic.
2. Weight and bias for the **hidden layer** are read from the **BRAM weight**, then sent to the **moving input** of the systolic.
3. Output from the systolic is processed by the **sigmoid module**, and then the result is sent to the **stationary input** of the systolic.

- Weight and bias for the **output layer** are read from the **BRAM weight**, then sent to the **moving input** of the systolic.
- Output from the systolic is processed by the **sigmoid module**, and then the result is sent to the **BRAM output**.

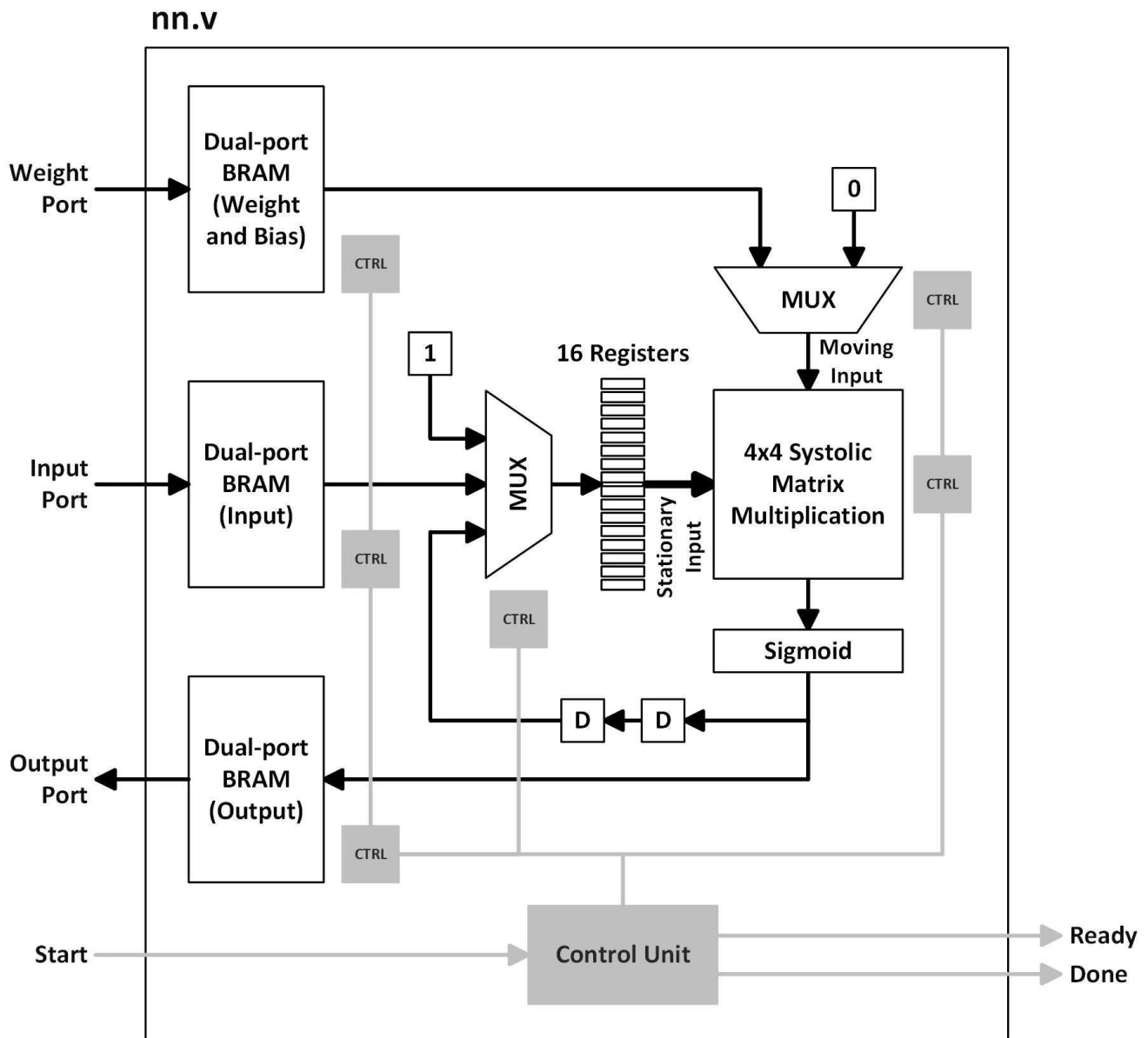


Figure 12. NN module block diagram

A start signal is used to start the NN module. A done signal is used to indicate that the NN computation is finished. During NN computation, the ready signal will be zero, which gives an indication that the NN module is busy.

This is the Verilog implementation of the NN module.

nn.v

```
`timescale 1ns / 1ps
```

```
module nn
```

```
(
```

```

    input wire      clk,
    input wire      rst_n,
    input wire      en,
    input wire      clr,
    // *** Control and status port ***
    output wire     ready,
    input wire      start,
    output wire     done,
    // *** Weight port ***
    input wire      wb_ena,
    input wire [2:0] wb_addra,
    input wire [63:0] wb_dina,
    input wire [7:0]  wb_wea,
    // *** Data input port ***
    input wire      k_ena,
    input wire [1:0] k_addra,
    input wire [63:0] k_dina,
    input wire [7:0]  k_wea,
    // *** Data output port ***
    input wire      a_enb,
    input wire [1:0] a_addrb,
    output wire [63:0] a_doutb
);

```

```

// Weight BRAM

```

```

wire wb_enb;
wire [2:0] wb_addrb;
wire [63:0] wb_doutb;

```

```

wire [15:0] wb_doutb_0;
wire [15:0] wb_doutb_1;
wire [15:0] wb_doutb_2;
wire [15:0] wb_doutb_3;

```

```

// Input BRAM

```

```

wire k_enb;
wire [1:0] k_addrb;
wire [63:0] k_doutb;

```

```

wire [15:0] k_doutb_0;
wire [15:0] k_doutb_1;
wire [15:0] k_doutb_2;
wire [15:0] k_doutb_3;

```

```

// Counter for main controller
reg [5:0] cnt_main_reg;

```

```

// Multiplexer and register for systolic moving input
wire [0:0] a0_sel, a1_sel, a2_sel, a3_sel;
wire [15:0] a0, a1, a2, a3;

// Multiplexer and register for systolic stationary input
wire [1:0] b00_sel, b01_sel, b02_sel, b03_sel;
wire [1:0] b10_sel, b11_sel, b12_sel, b13_sel;
wire [1:0] b20_sel, b21_sel, b22_sel, b23_sel;
wire [1:0] b30_sel, b31_sel, b32_sel, b33_sel;

wire [15:0] b00_next, b01_next, b02_next, b03_next;
wire [15:0] b10_next, b11_next, b12_next, b13_next;
wire [15:0] b20_next, b21_next, b22_next, b23_next;
wire [15:0] b30_next, b31_next, b32_next, b33_next;

wire [15:0] b00_reg, b01_reg, b02_reg, b03_reg;
wire [15:0] b10_reg, b11_reg, b12_reg, b13_reg;
wire [15:0] b20_reg, b21_reg, b22_reg, b23_reg;
wire [15:0] b30_reg, b31_reg, b32_reg, b33_reg;

// Systolic
wire sys_in_valid;
wire [15:0] y0, y1, y2, y3;
wire sys_out_valid;

// Sigmoid
wire [15:0] s0, s1, s2, s3;
wire sig_out_valid;

wire [15:0] s0_reg0, s0_reg1;
wire [15:0] s1_reg0, s1_reg1;
wire [15:0] s2_reg0, s2_reg1;
wire [15:0] s3_reg0, s3_reg1;

wire sig_out_valid_reg0, sig_out_valid_reg1;

// Output BRAM
wire a_ena;
wire [7:0] a_wea;
wire [1:0] a_addra;
wire [63:0] a_dina;

// *** Weight BRAM *****
// xpm_memory_tdprom: True Dual Port RAM
// Xilinx Parameterized Macro, version 2018.3

```

xpm_memory_tdpram

#{

```
// Common module parameters
.MEMORY_SIZE(512),           // DECIMAL, size: 8x64bit= 512 bits
.MEMORY_PRIMITIVE("auto"),   // String
.CLOCKING_MODE("common_clock"), // String, "common_clock"
.MEMORY_INIT_FILE("none"),   // String
.MEMORY_INIT_PARAM("0"),     // String
.USE_MEM_INIT(1),            // DECIMAL
.WAKEUP_TIME("disable_sleep"), // String
.MESSAGE_CONTROL(0),         // DECIMAL
.AUTO_SLEEP_TIME(0),         // DECIMAL
.ECC_MODE("no_ecc"),         // String
.MEMORY_OPTIMIZATION("true"), // String
.USE_EMBEDDED_CONSTRAINT(0), // DECIMAL

// Port A module parameters
.WRITE_DATA_WIDTH_A(64),     // DECIMAL, data width: 64-bit
.READ_DATA_WIDTH_A(64),      // DECIMAL, data width: 64-bit
.BYTE_WRITE_WIDTH_A(8),      // DECIMAL
.ADDR_WIDTH_A(3),            // DECIMAL,  $\text{clog}_2(512/64)=\text{clog}_2(8)= 3$ 
.READ_RESET_VALUE_A("0"),    // String
.READ_LATENCY_A(1),          // DECIMAL
.WRITE_MODE_A("write_first"), // String
.RST_MODE_A("SYNC"),         // String

// Port B module parameters
.WRITE_DATA_WIDTH_B(64),     // DECIMAL, data width: 64-bit
.READ_DATA_WIDTH_B(64),      // DECIMAL, data width: 64-bit
.BYTE_WRITE_WIDTH_B(8),      // DECIMAL
.ADDR_WIDTH_B(3),            // DECIMAL,  $\text{clog}_2(512/64)=\text{clog}_2(8)= 3$ 
.READ_RESET_VALUE_B("0"),    // String
.READ_LATENCY_B(1),          // DECIMAL
.WRITE_MODE_B("write_first"), // String
.RST_MODE_B("SYNC")         // String
```

)

xpm_memory_tdpram_wb

(

```
.sleep(1'b0),
.regcea(1'b1), //do not change
.injectsbitterra(1'b0), //do not change
.injectdbitterra(1'b0), //do not change
.sbiterra(), //do not change
.dbiterra(), //do not change
.regceb(1'b1), //do not change
.injectsbitterrb(1'b0), //do not change
```

```

.injectdbiterrb(1'b0), //do not change
.sbiterrb(), //do not change
.dbiterrb(), //do not change

// Port A module ports
.clka(clk),
.rsta(~rst_n),
.ena(wb_ena),
.wea(wb_wea),
.addra(wb_addra),
.dina(wb_dina),
.douta(),

// Port B module ports
.clkb(clk),
.rstb(~rst_n),
.enb(wb_enb),
.web(0),
.addrb(wb_addrb),
.dinb(0),
.doutb(wb_doutb)
);
assign wb_doutb_0 = wb_doutb[15:0];
assign wb_doutb_1 = wb_doutb[31:16];
assign wb_doutb_2 = wb_doutb[47:32];
assign wb_doutb_3 = wb_doutb[63:48];

// *** Input BRAM *****
// xpm_memory_tdpam: True Dual Port RAM
// Xilinx Parameterized Macro, version 2018.3
xpm_memory_tdpam
#(
    // Common module parameters
    .MEMORY_SIZE(256),                // DECIMAL, size: 4x64bit= 256 bits
    .MEMORY_PRIMITIVE("auto"),        // String
    .CLOCKING_MODE("common_clock"),   // String, "common_clock"
    .MEMORY_INIT_FILE("none"),        // String
    .MEMORY_INIT_PARAM("0"),          // String
    .USE_MEM_INIT(1),                 // DECIMAL
    .WAKEUP_TIME("disable_sleep"),    // String
    .MESSAGE_CONTROL(0),              // DECIMAL
    .AUTO_SLEEP_TIME(0),              // DECIMAL
    .ECC_MODE("no_ecc"),              // String
    .MEMORY_OPTIMIZATION("true"),     // String
    .USE_EMBEDDED_CONSTRAINT(0),      // DECIMAL

```

```

// Port A module parameters
.WRITE_DATA_WIDTH_A(64),           // DECIMAL, data width: 64-bit
.READ_DATA_WIDTH_A(64),           // DECIMAL, data width: 64-bit
.BYTE_WRITE_WIDTH_A(8),           // DECIMAL
.ADDR_WIDTH_A(2),                 // DECIMAL,  $\text{clog}_2(256/64)=\text{clog}_2(4)= 2$ 
.READ_RESET_VALUE_A("0"),        // String
.READ_LATENCY_A(1),              // DECIMAL
.WRITE_MODE_A("write_first"),     // String
.RST_MODE_A("SYNC"),             // String

// Port B module parameters
.WRITE_DATA_WIDTH_B(64),           // DECIMAL, data width: 64-bit
.READ_DATA_WIDTH_B(64),           // DECIMAL, data width: 64-bit
.BYTE_WRITE_WIDTH_B(8),           // DECIMAL
.ADDR_WIDTH_B(2),                 // DECIMAL,  $\text{clog}_2(256/64)=\text{clog}_2(8)= 2$ 
.READ_RESET_VALUE_B("0"),        // String
.READ_LATENCY_B(1),              // DECIMAL
.WRITE_MODE_B("write_first"),     // String
.RST_MODE_B("SYNC")              // String
)
xpm_memory_tdpram_k
(
    .sleep(1'b0),
    .regcea(1'b1), //do not change
    .injectsbitterra(1'b0), //do not change
    .injectdbitterra(1'b0), //do not change
    .sbitterra(), //do not change
    .dbitterra(), //do not change
    .regceb(1'b1), //do not change
    .injectsbitterrb(1'b0), //do not change
    .injectdbitterrb(1'b0), //do not change
    .sbitterrb(), //do not change
    .dbitterrb(), //do not change

    // Port A module ports
    .clk_a(clk),
    .rsta(~rst_n),
    .ena(k_ena),
    .wea(k_wea),
    .addra(k_addra),
    .dina(k_dina),
    .douta(),

    // Port B module ports
    .clk_b(clk),
    .rstb(~rst_n),

```

```

        .enb(k_enb),
        .web(0),
        .addrb(k_addrb),
        .dinb(0),
        .doutb(k_doutb)
    );
    assign k_doutb_0 = k_doutb[15:0];
    assign k_doutb_1 = k_doutb[31:16];
    assign k_doutb_2 = k_doutb[47:32];
    assign k_doutb_3 = k_doutb[63:48];

// *** Counter for main controller *****
always @(posedge clk)
begin
    if (!rst_n || clr)
    begin
        cnt_main_reg <= 0;
    end
    else if (start)
    begin
        cnt_main_reg <= cnt_main_reg + 1;
    end
    else if (cnt_main_reg >= 1 && cnt_main_reg <= 32)
    begin
        cnt_main_reg <= cnt_main_reg + 1;
    end
    else if (cnt_main_reg >= 33)
    begin
        cnt_main_reg <= 0;
    end
end

// Weight BRAM control
assign wb_enb = ((cnt_main_reg >= 3) && (cnt_main_reg <= 5)) ? 1 :
                ((cnt_main_reg >= 18) && (cnt_main_reg <= 19)) ? 1 : 0;
assign wb_addrb = (cnt_main_reg == 3) ? 0 :
                  (cnt_main_reg == 4) ? 1 :
                  (cnt_main_reg == 5) ? 2 :
                  (cnt_main_reg == 18) ? 3 :
                  (cnt_main_reg == 19) ? 4 : 0;

// Systolic moving input multiplexer control
assign a0_sel = ((cnt_main_reg >= 4) && (cnt_main_reg <= 6)) ? 0 :
                ((cnt_main_reg >= 19) && (cnt_main_reg <= 20)) ? 0 : 1;
assign a1_sel = ((cnt_main_reg >= 4) && (cnt_main_reg <= 6)) ? 0 :
                ((cnt_main_reg >= 19) && (cnt_main_reg <= 20)) ? 0 : 1;

```

```

assign a2_sel = ((cnt_main_reg >= 4) && (cnt_main_reg <= 6)) ? 0 :
                ((cnt_main_reg >= 19) && (cnt_main_reg <= 20)) ? 0 : 1;
assign a3_sel = ((cnt_main_reg >= 4) && (cnt_main_reg <= 6)) ? 0 :
                ((cnt_main_reg >= 19) && (cnt_main_reg <= 20)) ? 0 : 1;

// Input BRAM control
assign k_enb = ((cnt_main_reg >= 1) && (cnt_main_reg <= 2)) ? 1 : 0;
assign k_addrb = (cnt_main_reg == 1) ? 0 :
                 (cnt_main_reg == 2) ? 1 : 0;

// Systolic stationary input multiplexer control
assign b00_sel = (cnt_main_reg == 2) ? 0 :
                 (cnt_main_reg == 16) ? 1 : 3;
assign b01_sel = (cnt_main_reg == 2) ? 0 :
                 (cnt_main_reg == 16) ? 1 : 3;
assign b02_sel = (cnt_main_reg == 2) ? 0 :
                 (cnt_main_reg == 16) ? 1 : 3;
assign b03_sel = (cnt_main_reg == 2) ? 0 :
                 (cnt_main_reg == 16) ? 1 : 3;

assign b10_sel = (cnt_main_reg == 3) ? 0 :
                 (cnt_main_reg == 17) ? 1 : 3;
assign b11_sel = (cnt_main_reg == 3) ? 0 :
                 (cnt_main_reg == 17) ? 1 : 3;
assign b12_sel = (cnt_main_reg == 3) ? 0 :
                 (cnt_main_reg == 17) ? 1 : 3;
assign b13_sel = (cnt_main_reg == 3) ? 0 :
                 (cnt_main_reg == 17) ? 1 : 3;

assign b20_sel = (cnt_main_reg == 2) ? 2 :
                 (cnt_main_reg == 18) ? 1 : 3;
assign b21_sel = (cnt_main_reg == 2) ? 2 :
                 (cnt_main_reg == 18) ? 1 : 3;
assign b22_sel = (cnt_main_reg == 2) ? 2 :
                 (cnt_main_reg == 18) ? 1 : 3;
assign b23_sel = (cnt_main_reg == 2) ? 2 :
                 (cnt_main_reg == 18) ? 1 : 3;

assign b30_sel = (cnt_main_reg == 16) ? 2 : 3;
assign b31_sel = (cnt_main_reg == 16) ? 2 : 3;
assign b32_sel = (cnt_main_reg == 16) ? 2 : 3;
assign b33_sel = (cnt_main_reg == 16) ? 2 : 3;

// Systolic control
assign sys_in_valid = ((cnt_main_reg >= 4) && (cnt_main_reg <= 7)) ? 1 :
                      ((cnt_main_reg >= 19) && (cnt_main_reg <= 22)) ? 1 : 0;

```

```

// Output BRAM control
assign a_ena = ((cnt_main_reg >= 29) && (cnt_main_reg <= 30)) ? 1 : 0;
assign a_wea = ((cnt_main_reg >= 29) && (cnt_main_reg <= 30)) ? 8'hff : 0;
assign a_addra = (cnt_main_reg == 29) ? 0 :
                (cnt_main_reg == 30) ? 1 : 0;

// Status control
assign ready = (cnt_main_reg == 0) ? 1 : 0;
assign done = (cnt_main_reg == 33) ? 1 : 0;

// *** Multiplexer for systolic moving input *****
assign a0 = (a0_sel == 0) ? wb_doutb_0 : 0;
assign a1 = (a1_sel == 0) ? wb_doutb_1 : 0;
assign a2 = (a2_sel == 0) ? wb_doutb_2 : 0;
assign a3 = (a3_sel == 0) ? wb_doutb_3 : 0;

// *** Multiplexer and register for systolic stationary input *****
assign b00_next = (b00_sel == 0) ? k_doutb_0 :
                  (b00_sel == 1) ? s0_reg1 :
                  (b00_sel == 2) ? 16'b0000010000000000 : b00_reg;
assign b01_next = (b01_sel == 0) ? k_doutb_1 :
                  (b01_sel == 1) ? s1_reg1 :
                  (b01_sel == 2) ? 16'b0000010000000000 : b01_reg;
assign b02_next = (b02_sel == 0) ? k_doutb_2 :
                  (b02_sel == 1) ? s2_reg1 :
                  (b02_sel == 2) ? 16'b0000010000000000 : b02_reg;
assign b03_next = (b03_sel == 0) ? k_doutb_3 :
                  (b03_sel == 1) ? s3_reg1 :
                  (b03_sel == 2) ? 16'b0000010000000000 : b03_reg;

register #(16) reg_b00(clk, rst_n, en, clr, b00_next, b00_reg);
register #(16) reg_b01(clk, rst_n, en, clr, b01_next, b01_reg);
register #(16) reg_b02(clk, rst_n, en, clr, b02_next, b02_reg);
register #(16) reg_b03(clk, rst_n, en, clr, b03_next, b03_reg);

assign b10_next = (b10_sel == 0) ? k_doutb_0 :
                  (b10_sel == 1) ? s0_reg1 :
                  (b10_sel == 2) ? 16'b0000010000000000 : b10_reg;
assign b11_next = (b11_sel == 0) ? k_doutb_1 :
                  (b11_sel == 1) ? s1_reg1 :
                  (b11_sel == 2) ? 16'b0000010000000000 : b11_reg;
assign b12_next = (b12_sel == 0) ? k_doutb_2 :
                  (b12_sel == 1) ? s2_reg1 :
                  (b12_sel == 2) ? 16'b0000010000000000 : b12_reg;
assign b13_next = (b13_sel == 0) ? k_doutb_3 :

```

```

        (b13_sel == 1) ? s3_reg1 :
        (b13_sel == 2) ? 16'b0000010000000000 : b13_reg;

register #(16) reg_b10(clk, rst_n, en, clr, b10_next, b10_reg);
register #(16) reg_b11(clk, rst_n, en, clr, b11_next, b11_reg);
register #(16) reg_b12(clk, rst_n, en, clr, b12_next, b12_reg);
register #(16) reg_b13(clk, rst_n, en, clr, b13_next, b13_reg);

assign b20_next = (b20_sel == 0) ? k_doutb_0 :
        (b20_sel == 1) ? s0_reg1 :
        (b20_sel == 2) ? 16'b0000010000000000 : b20_reg;
assign b21_next = (b21_sel == 0) ? k_doutb_1 :
        (b21_sel == 1) ? s1_reg1 :
        (b21_sel == 2) ? 16'b0000010000000000 : b21_reg;
assign b22_next = (b22_sel == 0) ? k_doutb_2 :
        (b22_sel == 1) ? s2_reg1 :
        (b22_sel == 2) ? 16'b0000010000000000 : b22_reg;
assign b23_next = (b23_sel == 0) ? k_doutb_3 :
        (b23_sel == 1) ? s3_reg1 :
        (b23_sel == 2) ? 16'b0000010000000000 : b23_reg;

register #(16) reg_b20(clk, rst_n, en, clr, b20_next, b20_reg);
register #(16) reg_b21(clk, rst_n, en, clr, b21_next, b21_reg);
register #(16) reg_b22(clk, rst_n, en, clr, b22_next, b22_reg);
register #(16) reg_b23(clk, rst_n, en, clr, b23_next, b23_reg);

assign b30_next = (b30_sel == 0) ? k_doutb_0 :
        (b30_sel == 1) ? s0_reg1 :
        (b30_sel == 2) ? 16'b0000010000000000 : b30_reg;
assign b31_next = (b31_sel == 0) ? k_doutb_1 :
        (b31_sel == 1) ? s1_reg1 :
        (b31_sel == 2) ? 16'b0000010000000000 : b31_reg;
assign b32_next = (b32_sel == 0) ? k_doutb_2 :
        (b32_sel == 1) ? s2_reg1 :
        (b32_sel == 2) ? 16'b0000010000000000 : b32_reg;
assign b33_next = (b33_sel == 0) ? k_doutb_3 :
        (b33_sel == 1) ? s3_reg1 :
        (b33_sel == 2) ? 16'b0000010000000000 : b33_reg;

register #(16) reg_b30(clk, rst_n, en, clr, b30_next, b30_reg);
register #(16) reg_b31(clk, rst_n, en, clr, b31_next, b31_reg);
register #(16) reg_b32(clk, rst_n, en, clr, b32_next, b32_reg);
register #(16) reg_b33(clk, rst_n, en, clr, b33_next, b33_reg);

// *** Systolic *****
systolic

```

```

#(
    .WIDTH(16),
    .FRAC_BIT(10)
)
dut
(
    .clk(clk),
    .rst_n(rst_n),
    .en(en),
    .clr(clr),
    .a0(a0), .a1(a1), .a2(a2), .a3(a3),
    .in_valid(sys_in_valid),
    .b00(b00_reg), .b01(b01_reg), .b02(b02_reg), .b03(b03_reg),
    .b10(b10_reg), .b11(b11_reg), .b12(b12_reg), .b13(b13_reg),
    .b20(b20_reg), .b21(b21_reg), .b22(b22_reg), .b23(b23_reg),
    .b30(b30_reg), .b31(b31_reg), .b32(b32_reg), .b33(b33_reg),
    .y0(y0), .y1(y1), .y2(y2), .y3(y3),
    .out_valid(sys_out_valid)
);

// *** Sigmoid *****
sigmoid sigmoid_0(clk, rst_n, en, clr, y0, s0);
sigmoid sigmoid_1(clk, rst_n, en, clr, y1, s1);
sigmoid sigmoid_2(clk, rst_n, en, clr, y2, s2);
sigmoid sigmoid_3(clk, rst_n, en, clr, y3, s3);

register #(16) reg_sig_00(clk, rst_n, en, clr, s0,      s0_reg0);
register #(16) reg_sig_01(clk, rst_n, en, clr, s0_reg0, s0_reg1);
register #(16) reg_sig_10(clk, rst_n, en, clr, s1,      s1_reg0);
register #(16) reg_sig_11(clk, rst_n, en, clr, s1_reg0, s1_reg1);
register #(16) reg_sig_20(clk, rst_n, en, clr, s2,      s2_reg0);
register #(16) reg_sig_21(clk, rst_n, en, clr, s2_reg0, s2_reg1);
register #(16) reg_sig_30(clk, rst_n, en, clr, s3,      s3_reg0);
register #(16) reg_sig_31(clk, rst_n, en, clr, s3_reg0, s3_reg1);

register #(1) reg_sig_valid_0(clk, rst_n, en, clr, sys_out_valid,      sig_out_valid);
register #(1) reg_sig_valid_1(clk, rst_n, en, clr, sig_out_valid,
sig_out_valid_reg0);
    register #(1) reg_sig_valid_2(clk, rst_n, en, clr, sig_out_valid_reg0,
sig_out_valid_reg1);

// *** Output BRAM *****
assign a_dina = {s3, s2, s1, s0};
// xpm_memory_tdpram: True Dual Port RAM
// Xilinx Parameterized Macro, version 2018.3
xpm_memory_tdpram

```

```

#(
    // Common module parameters
    .MEMORY_SIZE(256),                // DECIMAL, size: 4x64bit= 256 bits
    .MEMORY_PRIMITIVE("auto"),        // String
    .CLOCKING_MODE("common_clock"),   // String, "common_clock"
    .MEMORY_INIT_FILE("none"),        // String
    .MEMORY_INIT_PARAM("0"),          // String
    .USE_MEM_INIT(1),                 // DECIMAL
    .WAKEUP_TIME("disable_sleep"),    // String
    .MESSAGE_CONTROL(0),              // DECIMAL
    .AUTO_SLEEP_TIME(0),              // DECIMAL
    .ECC_MODE("no_ecc"),              // String
    .MEMORY_OPTIMIZATION("true"),     // String
    .USE_EMBEDDED_CONSTRAINT(0),      // DECIMAL

    // Port A module parameters
    .WRITE_DATA_WIDTH_A(64),          // DECIMAL, data width: 64-bit
    .READ_DATA_WIDTH_A(64),           // DECIMAL, data width: 64-bit
    .BYTE_WRITE_WIDTH_A(8),           // DECIMAL
    .ADDR_WIDTH_A(2),                 // DECIMAL,  $\text{clog}_2(256/64)=\text{clog}_2(4)= 2$ 
    .READ_RESET_VALUE_A("0"),         // String
    .READ_LATENCY_A(1),               // DECIMAL
    .WRITE_MODE_A("write_first"),     // String
    .RST_MODE_A("SYNC"),              // String

    // Port B module parameters
    .WRITE_DATA_WIDTH_B(64),          // DECIMAL, data width: 64-bit
    .READ_DATA_WIDTH_B(64),           // DECIMAL, data width: 64-bit
    .BYTE_WRITE_WIDTH_B(8),           // DECIMAL
    .ADDR_WIDTH_B(2),                 // DECIMAL,  $\text{clog}_2(256/64)=\text{clog}_2(8)= 2$ 
    .READ_RESET_VALUE_B("0"),         // String
    .READ_LATENCY_B(1),               // DECIMAL
    .WRITE_MODE_B("write_first"),     // String
    .RST_MODE_B("SYNC")              // String
)
xpm_memory_tdpram_a
(
    .sleep(1'b0),
    .regcea(1'b1), //do not change
    .injectsbitterra(1'b0), //do not change
    .injectdbitterra(1'b0), //do not change
    .sbitterra(), //do not change
    .dbitterra(), //do not change
    .regceb(1'b1), //do not change
    .injectsbitterrb(1'b0), //do not change
    .injectdbitterrb(1'b0), //do not change

```

```

.sbiterrb(), //do not change
.dbiterrb(), //do not change

// Port A module ports
.clka(clk),
.rsta(~rst_n),
.ena(a_ena),
.wea(a_wea),
.addra(a_addra),
.dina(a_dina),
.douta(),

// Port B module ports
.clkb(clk),
.rstb(~rst_n),
.enb(a_enb),
.web(0),
.addrb(a_addrb),
.dinb(0),
.doutb(a_doutb)
);

endmodule

```

The following figures show how the data is stored inside the BRAM. The BRAM data width is 64-bit. For weight and bias, the data depth is 8, while for input and output, it is 4.

	63	...	48	47	...	32	31	...	16	15	...	0
0	0.00			-19.88			1.37			1.37		
1	0.00			-0.90			0.97			0.77		
2	0.00			-0.89			0.64			1.05		
3	-2.59			0.08			-1.31			7.11		
4	0.20			1.97			1.63			-7.10		
5	0.00			0.00			0.00			0.00		
6	0.00			0.00			0.00			0.00		
7	0.00			0.00			0.00			0.00		

**BRAM Weight and Bias
64-bit**

Figure 13. BRAM weight and bias

	63	...	48	47	...	32	31	...	16	15	...	0
0	5.00		5.00			8.00			8.00			
1	5.00		8.00			5.00			8.00			
2	0.00		0.00			0.00			0.00			
3	0.00		0.00			0.00			0.00			

**BRAM Input
64-bit**

Figure 14. BRAM input

	63	...	48	47	...	32	31	...	16	15	...	0
0	0.020		0.042			0.042			0.914			
1	0.977		0.952			0.952			0.075			
2	0.000		0.000			0.000			0.000			
3	0.000		0.000			0.000			0.000			

**BRAM Output
64-bit**

Figure 15. BRAM output

The following figures show the BRAM value in the Vivado simulation.

mem[0:7][63:0]	0000b07a057a057a
> [0][63:0]	0000b07a057a057a
> [1][63:0]	0000fc6603e10314
> [2][63:0]	0000fc70028f0433
> [3][63:0]	f5a30051fac21c70
> [4][63:0]	00cc07e10685e399
> [5][63:0]	0000000000000000
> [6][63:0]	0000000000000000
> [7][63:0]	0000000000000000

Figure 16. BRAM weight and bias in simulation

mem[0:3][63:0]	1400140020002000
> [0][63:0]	1400140020002000
> [1][63:0]	1400200014002000
> [2][63:0]	0000000000000000
> [3][63:0]	0000000000000000

Figure 17. BRAM input in simulation

mem[0:3][63:0]	0014002b002b03a8
> [0][63:0]	0014002b002b03a8
> [1][63:0]	03e803cf03cf004d
> [2][63:0]	0000000000000000
> [3][63:0]	0000000000000000

Figure 18. BRAM output in simulation

The following figure shows the timing diagram of the NN module. The module starts when the start signal is one. For the duration of the computation, the ready signal is zero, indicating that the NN module is busy.

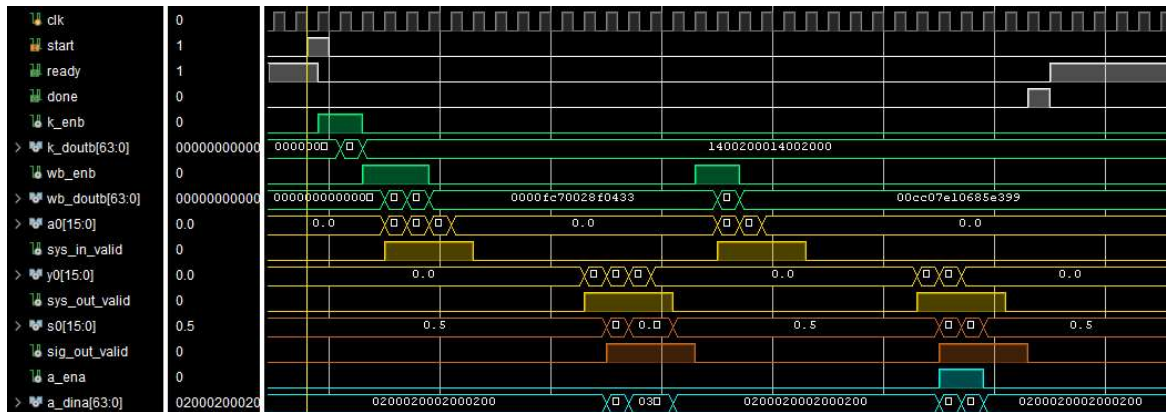


Figure 19. Timing diagram of NN module

First, the NN module reads the input and weight for the hidden layer. Then, it is processed with the systolic and sigmoid modules. The result is processed again with the output layer's weight. Finally, the final result is stored in the output BRAM.

Now, we already have the NN module. The next step is to connect these blocks to the standard interface that can be understood by the CPU. In this case, we use the AXI-Stream interface. This is how the data flows:

1. Both the **weight** and the **input** data are streamed through the **s_axis** port.
2. The **demultiplexer** separates which data goes to the weight port and which data goes to the input port of the NN module.
3. The control unit **starts** the NN module and waits until it is **finished**.
4. The **output** data is streamed out to the **m_axis** port.

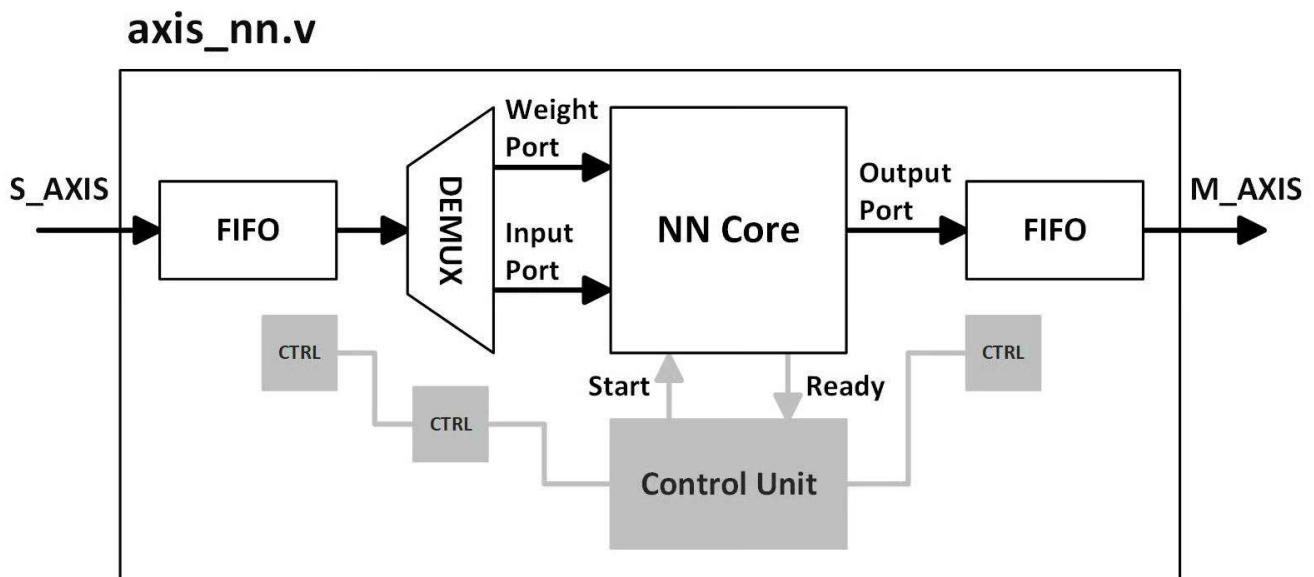


Figure 20. AXIS NN module block diagram

This is the Verilog implementation of the AXIS NN module.

axis_nn.v

```
`timescale 1ns / 1ps

module axis_nn
(
    input wire      aclk,
    input wire      aresetn,
    // *** AXIS slave port ***
    output wire      s_axis_tready,
    input wire [63:0] s_axis_tdata,
    input wire      s_axis_tvalid,
    input wire      s_axis_tlast,
    // *** AXIS master port ***
    input wire      m_axis_tready,
    output wire [63:0] m_axis_tdata,
    output wire      m_axis_tvalid,
    output wire      m_axis_tlast
);

// State machine
reg [2:0] state_reg, state_next;
reg [2:0] cnt_word_reg, cnt_word_next;

// MM2S FIFO
wire [8:0] mm2s_data_count;
wire start_from_mm2s;
reg mm2s_ready_reg, mm2s_ready_next;
wire [63:0] mm2s_data;

// NN
wire nn_start;
wire nn_ready;
wire wb_ena;
wire [2:0] wb_addra;
wire [63:0] wb_dina;
wire [7:0] wb_wea;
wire k_ena;
wire [1:0] k_addra;
wire [63:0] k_dina;
wire [7:0] k_wea;
wire a_enb;
wire [1:0] a_addrb;
wire [63:0] a_doutb;
```

```

// S2MM FIFO
wire s2mm_ready;
wire [63:0] s2mm_data;
wire s2mm_valid, s2mm_valid_reg;
wire s2mm_last, s2mm_last_reg;

// *** MM2S FIFO *****
// xpm_fifo_axis: AXI Stream FIFO
// Xilinx Parameterized Macro, version 2018.3
xpm_fifo_axis
#(
    .CDC_SYNC_STAGES(2),           // DECIMAL
    .CLOCKING_MODE("common_clock"), // String
    .ECC_MODE("no_ecc"),           // String
    .FIFO_DEPTH(256),              // DECIMAL, depth 256 elemen
    .FIFO_MEMORY_TYPE("auto"),     // String
    .PACKET_FIFO("false"),         // String
    .PROG_EMPTY_THRESH(10),        // DECIMAL
    .PROG_FULL_THRESH(10),         // DECIMAL
    .RD_DATA_COUNT_WIDTH(1),       // DECIMAL
    .RELATED_CLOCKS(0),            // DECIMAL
    .SIM_ASSERT_CHK(0),            // DECIMAL
    .TDATA_WIDTH(64),              // DECIMAL, data width 64 bit
    .TDEST_WIDTH(1),               // DECIMAL
    .TID_WIDTH(1),                 // DECIMAL
    .TUSER_WIDTH(1),               // DECIMAL
    .USE_ADV_FEATURES("0004"),     // String, write data count
    .WR_DATA_COUNT_WIDTH(9)        // DECIMAL, width log2(256)+1=9
)
xpm_fifo_axis_0
(
    .almost_empty_axis(),
    .almost_full_axis(),
    .dbiterr_axis(),
    .prog_empty_axis(),
    .prog_full_axis(),
    .rd_data_count_axis(),
    .sbiterr_axis(),
    .injectdbiterr_axis(1'b0),
    .injectsbiterr_axis(1'b0),

    .s_aclk(aclk), // aclk
    .m_aclk(aclk), // aclk
    .s_aresetn(aresetn), // aresetn

    .s_axis_tready(s_axis_tready), // ready

```

```

.s_axis_tdata(s_axis_tdata), // data
.s_axis_tvalid(s_axis_tvalid), // valid
.s_axis_tdest(1'b0),
.s_axis_tid(1'b0),
.s_axis_tkeep(8'hff),
.s_axis_tlast(s_axis_tlast),
.s_axis_tstrb(8'hff),
.s_axis_tuser(1'b0),

.m_axis_tready(mm2s_ready_reg), // ready
.m_axis_tdata(mm2s_data), // data
.m_axis_tvalid(), // valid
.m_axis_tdest(),
.m_axis_tid(),
.m_axis_tkeep(),
.m_axis_tlast(),
.m_axis_tstrb(),
.m_axis_tuser(),

.wr_data_count_axis(mm2s_data_count) // data count
);

// *** Main control *****
// Start signal from DMA MM2S
assign start_from_mm2s = (mm2s_data_count >= 7); // Weight = 5 word, input = 2 word,
total = 7 word

// State machine for AXI-Stream protocol
always @(posedge aclk)
begin
    if (!aresetn)
    begin
        state_reg <= 0;
        mm2s_ready_reg <= 0;
        cnt_word_reg <= 0;
    end
    else
    begin
        state_reg <= state_next;
        mm2s_ready_reg <= mm2s_ready_next;
        cnt_word_reg <= cnt_word_next;
    end
end

always @(*)
begin

```

```

state_next = state_reg;
mm2s_ready_next = mm2s_ready_reg;
cnt_word_next = cnt_word_reg;
case (state_reg)
    0: // Wait until data from MM2S is ready (7 words)
        begin
            if (start_from_mm2s)
                begin
                    state_next = 1;
                    mm2s_ready_next = 1; // Tell the MM2S FIFO that it is ready to accept
data
                    end
                end
            end
        1: // Write data to weight BRAM of the NN
            begin
                if (cnt_word_reg == 4)
                    begin
                        state_next = 2;
                        cnt_word_next = 0;
                    end
                else
                    begin
                        cnt_word_next = cnt_word_reg + 1;
                    end
                end
            end
        2: // Write data to input BRAM of the NN
            begin
                if (cnt_word_reg == 1)
                    begin
                        state_next = 3;
                        mm2s_ready_next = 0;
                        cnt_word_next = 0;
                    end
                else
                    begin
                        cnt_word_next = cnt_word_reg + 1;
                    end
                end
            end
        3: // Start the NN
            begin
                state_next = 4;
            end
        4: // Wait until NN computation done and S2MM FIFO is ready to accept data
            begin
                if (nn_ready && s2mm_ready)
                    begin

```

```

        state_next = 5;
    end
end
5: // Read data output from BRAM of the NN
begin
    if (cnt_word_reg == 1)
    begin
        state_next = 0;
        cnt_word_next = 0;
    end
    else
    begin
        cnt_word_next = cnt_word_reg + 1;
    end
    end
endcase
end

// Control weight port NN
assign wb_ena = (state_reg == 1) ? 1 : 0;
assign wb_addra = cnt_word_reg;
assign wb_dina = mm2s_data;
assign wb_wea = (state_reg == 1) ? 8'hff : 0;

// Control data input port NN
assign k_ena = (state_reg == 2) ? 1 : 0;
assign k_addra = cnt_word_reg[1:0];
assign k_dina = mm2s_data;
assign k_wea = (state_reg == 2) ? 8'hff : 0;

// Start NN
assign nn_start = (state_reg == 3) ? 1 : 0;

// Control data output port NN
assign a_enb = (state_reg == 5) ? 1 : 0;
assign a_addrb = cnt_word_reg[1:0];

// Control S2MM FIFO
assign s2mm_data = a_doutb;
assign s2mm_valid = a_enb;
register #(1) reg_s2mm_valid(aclk, aresetn, 1'b1, 1'b0, s2mm_valid, s2mm_valid_reg);
assign s2mm_last = ((state_reg == 5) && (a_addrb == 2'b01)) ? 1 : 0;
register #(1) reg_s2mm_last(aclk, aresetn, 1'b1, 1'b0, s2mm_last, s2mm_last_reg);

// *** NN *****
nn_nn_0

```

```

(
    .clk(ac1k),
    .rst_n(aresetn),
    .en(1'b1),
    .clr(1'b0),
    .ready(nn_ready),
    .start(nn_start),
    .done(),
    .wb_ena(wb_ena),
    .wb_addra(wb_addra),
    .wb_dina(wb_dina),
    .wb_wea(wb_wea),
    .k_ena(k_ena),
    .k_addra(k_addra),
    .k_dina(k_dina),
    .k_wea(k_wea),
    .a_enb(a_enb),
    .a_addrb(a_addrb),
    .a_doutb(a_doutb)
);

// *** S2MM FIFO *****
// xpm_fifo_axis: AXI Stream FIFO
// Xilinx Parameterized Macro, version 2018.3
xpm_fifo_axis
#(
    .CDC_SYNC_STAGES(2),           // DECIMAL
    .CLOCKING_MODE("common_clock"), // String
    .ECC_MODE("no_ecc"),           // String
    .FIFO_DEPTH(256),              // DECIMAL, depth 256 elemen
    .FIFO_MEMORY_TYPE("auto"),      // String
    .PACKET_FIFO("false"),         // String
    .PROG_EMPTY_THRESH(10),        // DECIMAL
    .PROG_FULL_THRESH(10),         // DECIMAL
    .RD_DATA_COUNT_WIDTH(1),        // DECIMAL
    .RELATED_CLOCKS(0),            // DECIMAL
    .SIM_ASSERT_CHK(0),            // DECIMAL
    .TDATA_WIDTH(64),              // DECIMAL, data width 64 bit
    .TDEST_WIDTH(1),               // DECIMAL
    .TID_WIDTH(1),                 // DECIMAL
    .TUSER_WIDTH(1),               // DECIMAL
    .USE_ADV_FEATURES("0004"),     // String, write data count
    .WR_DATA_COUNT_WIDTH(9)        // DECIMAL, width log2(256)+1=9
)
xpm_fifo_axis_1
(

```

```

.almost_empty_axis(),
.almost_full_axis(),
.dbiterr_axis(),
.prog_empty_axis(),
.prog_full_axis(),
.rd_data_count_axis(),
.sbiterr_axis(),
.injectdbiterr_axis(1'b0),
.injectsbiterr_axis(1'b0),

.s_aclk(aclk), // aclk
.m_aclk(aclk), // aclk
.s_aresetn(aresetn), // aresetn

.s_axis_tready(s2mm_ready), // ready
.s_axis_tdata(s2mm_data), // data
.s_axis_tvalid(s2mm_valid_reg), // valid
.s_axis_tdest(1'b0),
.s_axis_tid(1'b0),
.s_axis_tkeep(8'hff),
.s_axis_tlast(s2mm_last_reg),
.s_axis_tstrb(8'hff),
.s_axis_tuser(1'b0),

.m_axis_tready(m_axis_tready), // ready
.m_axis_tdata(m_axis_tdata), // data
.m_axis_tvalid(m_axis_tvalid), // valid
.m_axis_tdest(),
.m_axis_tid(),
.m_axis_tkeep(),
.m_axis_tlast(m_axis_tlast),
.m_axis_tstrb(),
.m_axis_tuser(),

.wr_data_count_axis() // data count
);

```

endmodule

The following figure shows the timing diagram of the AXIS NN module. The control unit module starts when the number of received streams of data is 7.

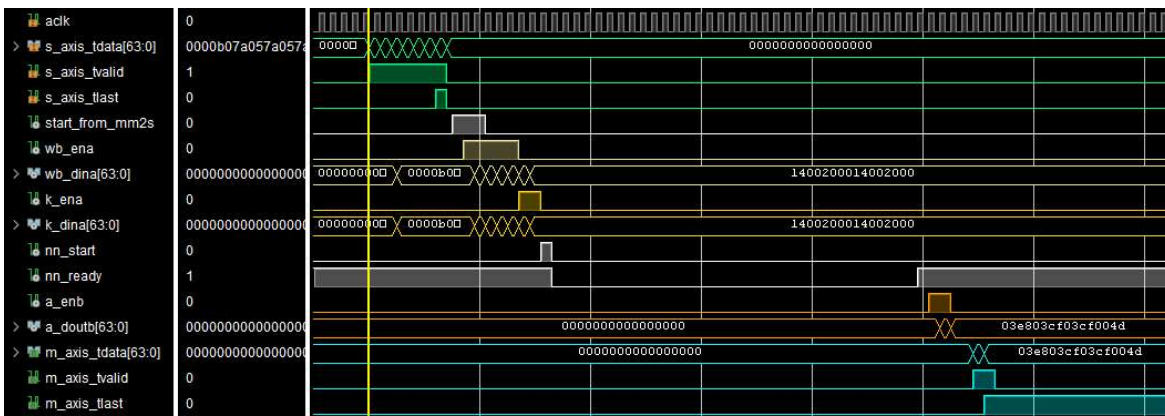


Figure 21. Timing diagram of AXIS NN module

The AXIS NN control module reads the data that is temporarily stored in FIFO and sends it to the NN module. Then, it starts the module, and it waits until it is finished. Then, the output data is temporarily stored in FIFO before being sent to the `m_AXIS` port.

The following figure shows the overall SoC system design for the NN accelerator. We use the AXI Streaming FIFO IP that converts memory-mapped data to stream data and vice versa. This method is the most basic conversion. Another method that can be used is the AXI DMA IP.

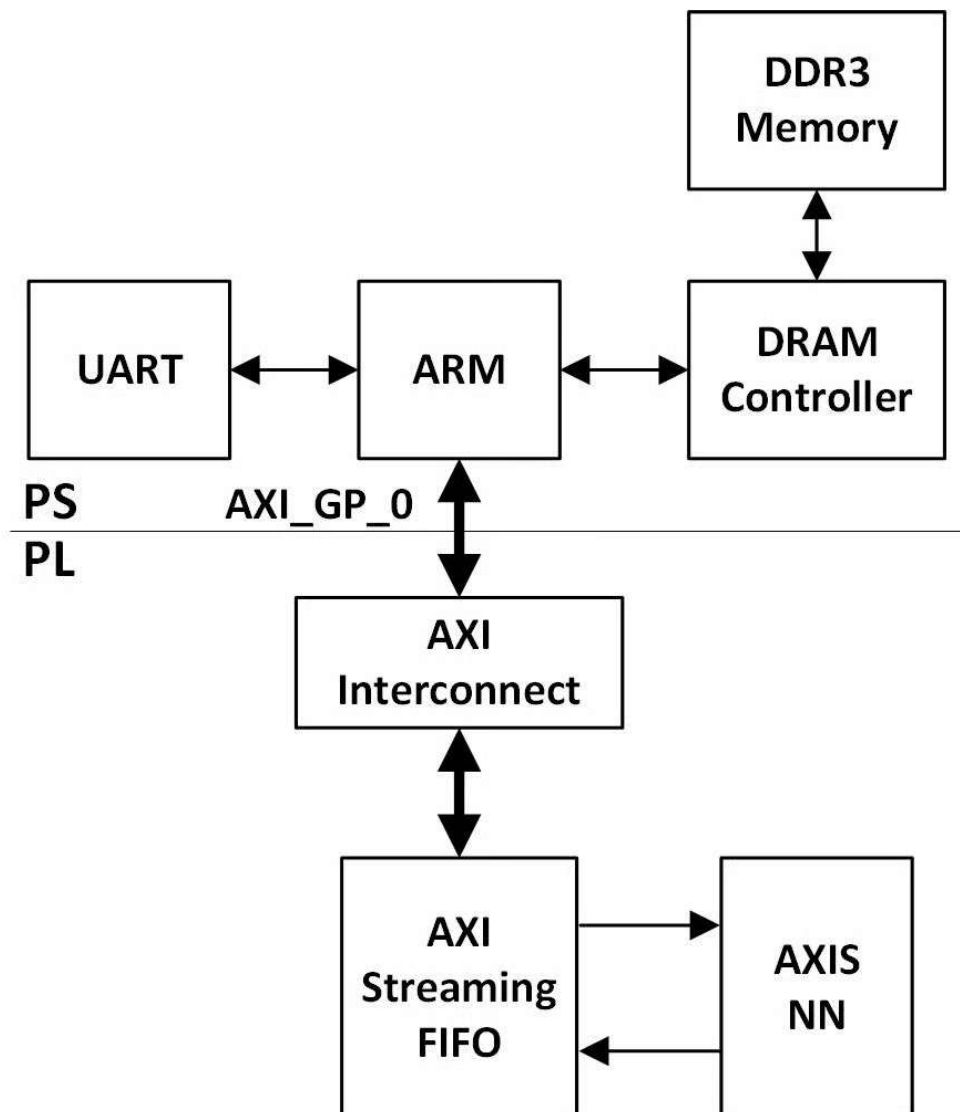


Figure 22. Block diagram of NN system

The following figure shows the block design in Vivado.

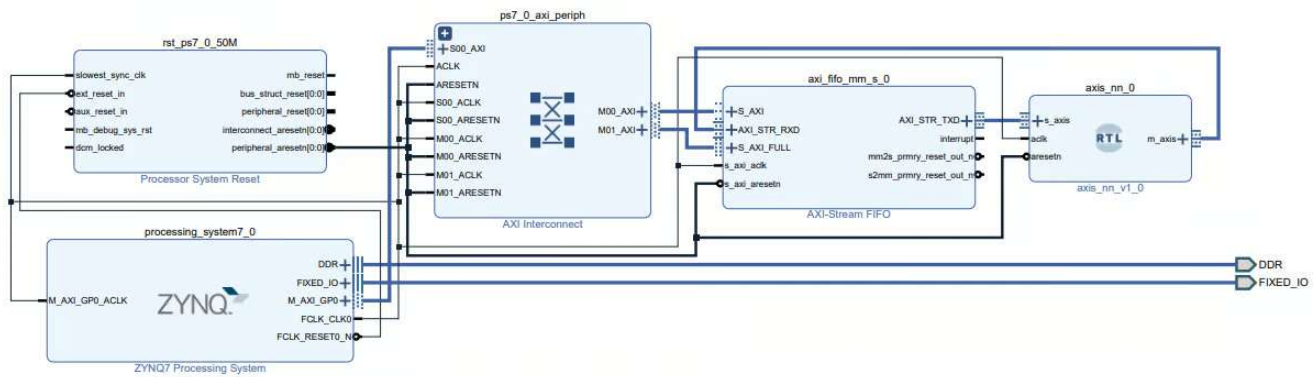


Figure 23. Block diagram of NN system in Vivado block design

For the software design, we use the SDK library for AXI Streaming FIFO IP. We need to declare an array for the source and destination. Then, we define `TxSend()` to send weight and input data to the NN module and `RxReceive()` to receive output data from the NN module.

The output data format is still in fixed point format, so we have converted it to a float by dividing it by 1024 (10-bit fractions).

helloworld.c

```
#include <stdio.h>
#include "xparameters.h"
#include "xllfifo.h"
#include "xstatus.h"

#define WORD_SIZE      8 // Size of words in bytes
#define INPUT_LEN      7 // Weight and input
#define OUTPUT_LEN     2 // Output

int Init_XLlFifo(XLlFifo *InstancePtr, u16 DeviceId);
int TxSend(XLlFifo *InstancePtr, u64 *SourceAddr);
int RxReceive(XLlFifo *InstancePtr, u64 *DestinationAddr);

XLlFifo FifoInstance;
u64 SourceBuffer[INPUT_LEN];
u64 DestinationBuffer[OUTPUT_LEN];
float out0[4], out1[4];

int main()
{
    // Initialize AXI Stream FIFO IP
    Init_XLlFifo(&FifoInstance, XPAR_AXI_FIFO_0_DEVICE_ID);

    printf("Initialization success\n");
```

```

// Weight
SourceBuffer[0] = 0x0000B07A057A057A;
SourceBuffer[1] = 0x0000FC6603E10314;
SourceBuffer[2] = 0x0000FC70028F0433;
SourceBuffer[3] = 0xF5A30051FAC21C70;
SourceBuffer[4] = 0x00CC07E10685E399;

// Input
SourceBuffer[5] = 0x1400140020002000;
SourceBuffer[6] = 0x1400200014002000;

// Check weight and input
printf("SourceBuffer:\n");
for (int i = 0; i < INPUT_LEN; i++)
    printf(" 0x%016lX\n", SourceBuffer[i]);

// Send to NN core
TxSend(&FifoInstance, SourceBuffer);

// Read from NN core
RxReceive(&FifoInstance, DestinationBuffer);

// Check output
printf("DestinationBuffer:\n");
for (int i = 0; i < OUTPUT_LEN; i++)
    printf(" 0x%016lX\n", DestinationBuffer[i]);

// Decode output (We use 10 fraction bits, so we divide by 2^10 = 1024)
out0[0] = (u16)((DestinationBuffer[0] & 0x000000000000FFFF) / 1024.0;
out1[0] = (u16)((DestinationBuffer[1] & 0x000000000000FFFF) / 1024.0;
out0[1] = (u16)((DestinationBuffer[0] & 0x00000000FFFF0000) >> 16) / 1024.0;
out1[1] = (u16)((DestinationBuffer[1] & 0x00000000FFFF0000) >> 16) / 1024.0;
out0[2] = (u16)((DestinationBuffer[0] & 0x0000FFFF00000000) >> 32) / 1024.0;
out1[2] = (u16)((DestinationBuffer[1] & 0x0000FFFF00000000) >> 32) / 1024.0;
out0[3] = (u16)((DestinationBuffer[0] & 0xFFFF000000000000) >> 48) / 1024.0;
out1[3] = (u16)((DestinationBuffer[1] & 0xFFFF000000000000) >> 48) / 1024.0;

// Print final output
printf("Final NN output:\n");
printf(" [%.3f, %.3f]\n", out0[0], out1[0]);
printf(" [%.3f, %.3f]\n", out0[1], out1[1]);
printf(" [%.3f, %.3f]\n", out0[2], out1[2]);
printf(" [%.3f, %.3f]\n", out0[3], out1[3]);

return 0;

```

```

}

```

```

int Init_XLlFifo(XLlFifo *InstancePtr, u16 DeviceId)
{
    XLlFifo_Config *Config;
    int Status;

    Config = XLlFifo_LookupConfig(DeviceId);
    if (!Config)
    {
        printf("No config found for %d\n", DeviceId);
        return XST_FAILURE;
    }

    Status = XLlFifo_CfgInitialize(InstancePtr, Config, Config->BaseAddress);
    if (Status != XST_SUCCESS)
    {
        printf("Initialization failed\n");
        return XST_FAILURE;
    }

    XLlFifo_IntClear(InstancePtr, 0xffffffff);
    Status = XLlFifo_Status(InstancePtr);
    if (Status != 0x0)
    {
        printf("Reset failed\n");
        return XST_FAILURE;
    }

    return XST_SUCCESS;
}

int TxSend(XLlFifo *InstancePtr, u64 *SourceAddr)
{
    // Writing into the FIFO transmit buffer
    for(int i = 0; i < INPUT_LEN; i++)
        if (XLlFifo_iTxVacancy(InstancePtr))
            Xil_Out64(InstancePtr->Axi4BaseAddress + XLlF_AXI4_TDFD_OFFSET, *
(SourceAddr+i));

    // Start transmission by writing transmission length into the TLR
    XLlFifo_iTxSetLen(InstancePtr, (INPUT_LEN * WORD_SIZE));

    // Check for transmission completion
    while (!(XLlFifo_IsTxDone(InstancePtr)));

    return XST_SUCCESS;
}

```

```

}

int RxReceive(XLlFifo *InstancePtr, u64* DestinationAddr)
{
    static u32 ReceiveLength;
    u64 RxWord;
    int Status;

    while (XLlFifo_iRxOccupancy(InstancePtr))
    {
        // Read receive length
        ReceiveLength = XLlFifo_iRxGetLen(InstancePtr) / WORD_SIZE;
        // Reading from the FIFO receive buffer
        for (int i = 0; i < ReceiveLength; i++)
        {
            RxWord = Xil_In64(InstancePtr->Axi4BaseAddress + XLLF_AXI4_RDFD_OFFSET);
            *(DestinationAddr+i) = RxWord;
        }
    }

    // Check for receive completion
    Status = XLlFifo_IsRxDone(InstancePtr);
    if (Status != TRUE)
    {
        printf("Failing in receive complete\n");
        return XST_FAILURE;
    }

    return XST_SUCCESS;
}

```

The following figure shows the result on the serial terminal. The result of the output layer is similar to our calculation before:

$$A_3 = \begin{bmatrix} 0.922 & 0.046 & 0.046 & 0.021 \\ 0.077 & 0.952 & 0.952 & 0.977 \end{bmatrix}$$

A3=[0.9220.0770.0460.9520.0460.9520.0210.977]

```
Connected to: Serial ( COM13, 115200, 0, 8 )
Initialization success
SourceBuffer:
0x0000B07A057A057A
0x0000FC6603E10314
0x0000FC70028F0433
0xF5A30051FAC21C70
0x00CC07E10685E399
0x1400140020002000
0x1400200014002000
DestinationBuffer:
0x0014002B002B03A8
0x03E803CF03CF004D
Final NN output:
[0.914, 0.075]
[0.042, 0.952]
[0.042, 0.952]
[0.020, 0.977]
```

Figure 24. NN module output printed on the serial terminal

In this tutorial, we covered the main project of a NN accelerator.

